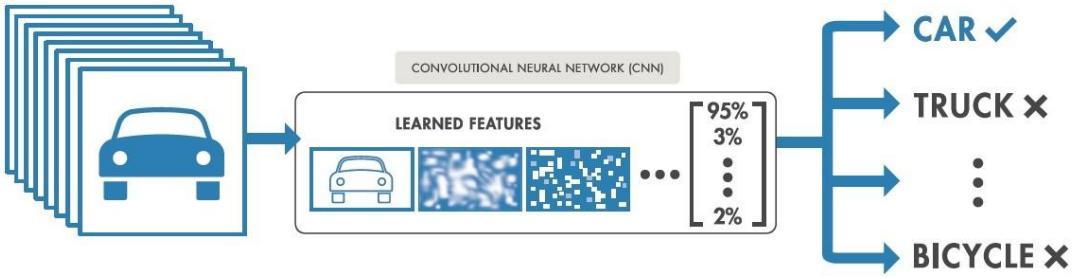
Deep Learning for Image Classification

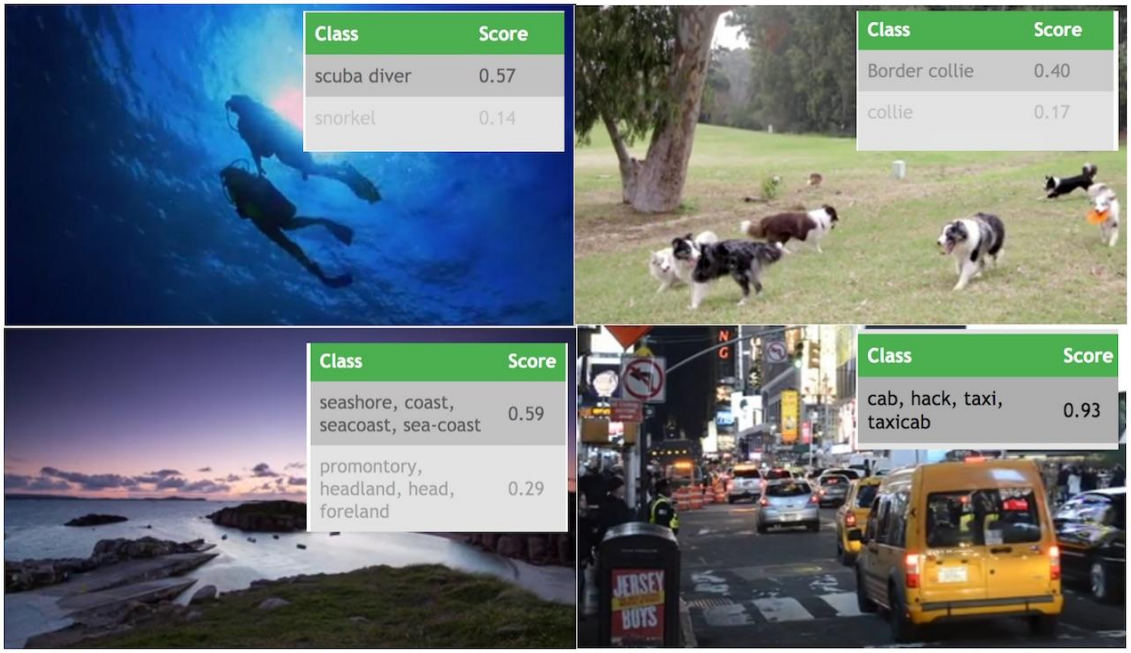


Matteo Ferrara
matteo.ferrara@unibo.it

Introduzione

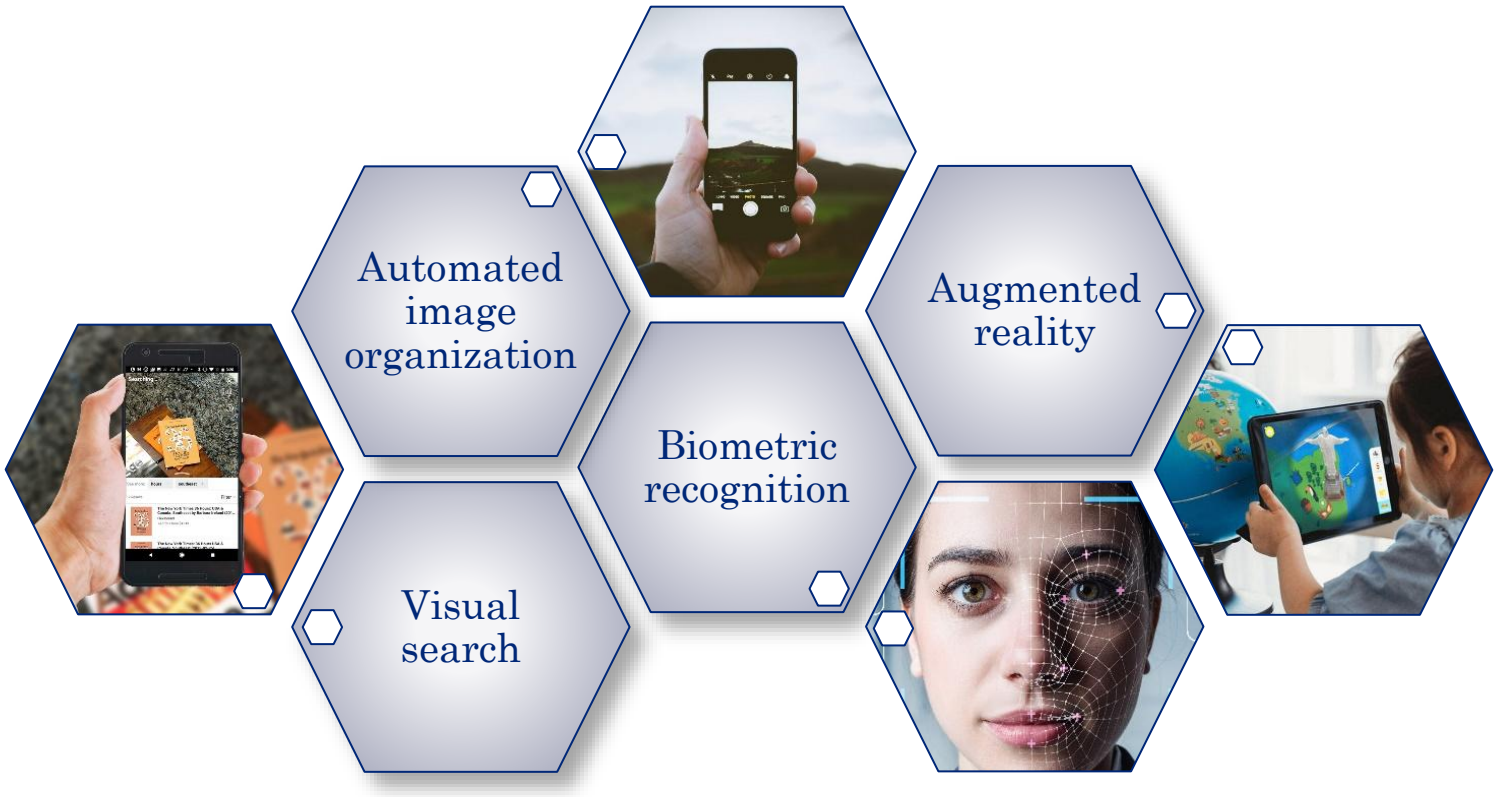
Data un'immagine di input, il processo di *image classification* (o *image recognition*) restituisce l'etichetta di una classe (scelta tra un insieme predefinito) e spesso una probabilità (o *score*) associata alla classe restituita.

Di solito la classe restituita si riferisce all'oggetto predominante nell'immagine analizzata.



Introduzione (2)

Da ormai una quindicina d'anni i problemi di *image classification* vengono sempre più spesso affrontati con l'uso di Convolutional Neural Network (CNN).



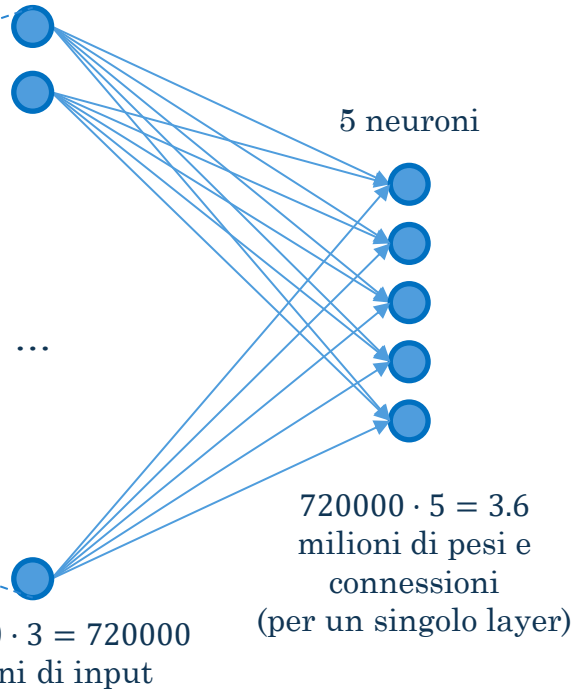
Svantaggi delle reti neurali tradizionali

Ci sono diversi svantaggi quando reti neurali tradizionali (MLP) vengono utilizzate per l'elaborazione delle immagini:

- Un neurone per ogni input - il numero di pesi cresce rapidamente e diventa ingestibile per immagini di grandi dimensioni.



600x400 pixel
3 canali (RGB)



Svantaggi delle reti neurali tradizionali (2)

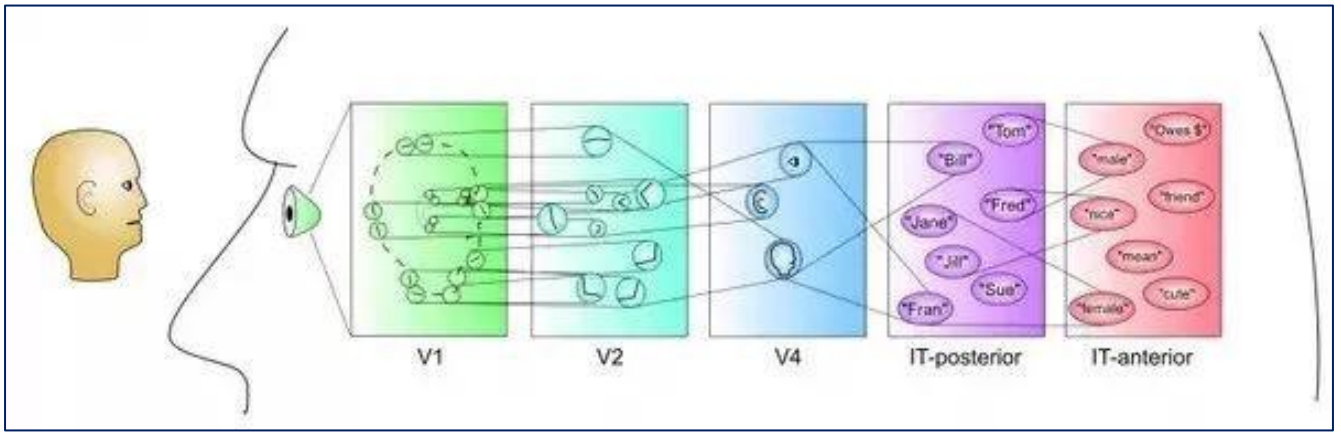
- I MLP non sono invarianti per traslazione - reagiscono in modo diverso a un'immagine e alla sua versione traslata.



- La relazione spaziale viene persa - pixel adiacenti sono fondamentali perché definiscono le caratteristiche dell'immagine.

La corteccia visiva

Nel 1968, Hubel e Wiesel (due neuro scienziati) hanno dimostrato che l'apparato visivo dei mammiferi presenta un'architettura a strati.



La struttura della corteccia visiva è stratificata.

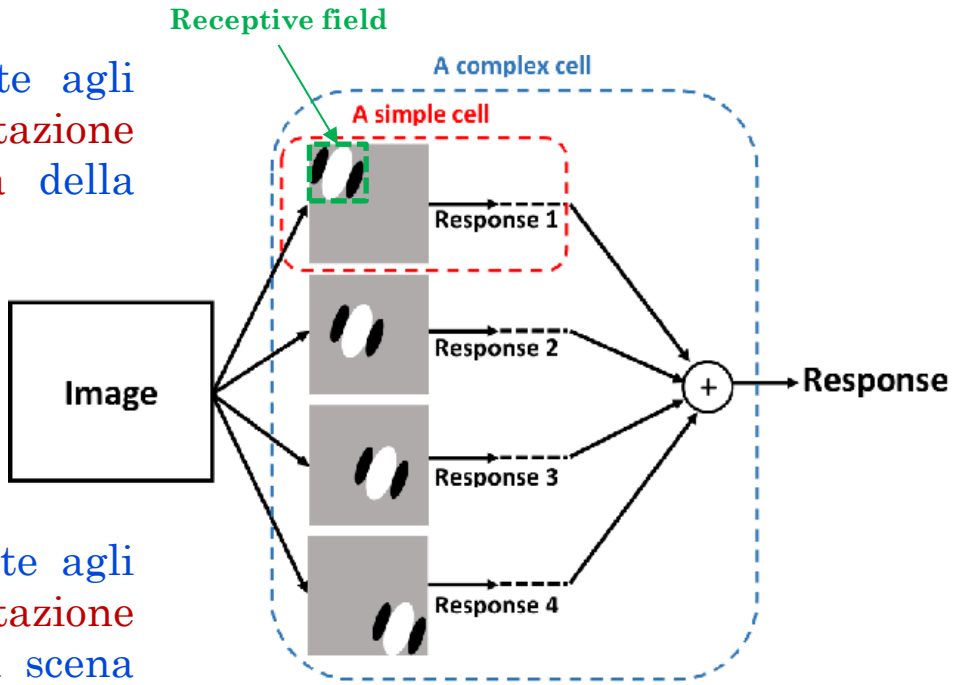
Quando le informazioni sono trasmesse dai nostri occhi al cervello, si formano rappresentazioni sempre più complesse mano a mano che le informazioni arrivano negli strati più profondi.

D. H. Hubel and T. N. Wiesel, "Receptive fields and functional architecture of monkey striate cortex," *Journal of Physiology (London)*, vol. 195, pp. 215–243, 1968.

La corteccia visiva (2)

Nella corteccia visiva umana, rappresentazioni complesse generate da *complex cell* sono realizzate combinando tra loro le risposte ottenute da *simple cell*:

- le simple cell rispondono solamente agli edge con una particolare orientazione presenti in una determinata zona della scena (*receptive field*);
- le complex cell rispondono solamente agli edge con una particolare orientazione indipendentemente dalla zona della scena in cui si trovano (invarianza spaziale).



La corteccia visiva (3)

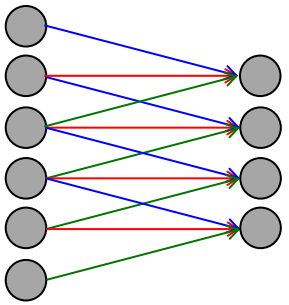
L'invarianza spaziale è ottenuta "sommando" il contributo di simple cell che rispondono tutte alla stessa orientazione ma con "campi visivi" (receptive field) diversi.

Il concetto che cellule semplici possano essere "combinate" per creare cellule più complesse è alla base del sistema visivo umano ma anche delle *Convolutional Neural Network* (CNN) usate negli odierni sistemi di *computer vision*.

Convolutional Neural Network

Le *Convolutional Neural Network* (CNN) sono state introdotte a partire dal 1998. Le principali differenze rispetto a un *Multi-Layer Perceptron* (MLP) sono:

1) processing locale - i neuroni sono connessi solo localmente ai neuroni del livello precedente. Forte riduzione numero di connessioni;
2) pesi condivisi - i pesi sono condivisi a gruppi. Neuroni diversi dello stesso livello eseguono lo stesso tipo di elaborazione su porzioni diverse dell'input. Forte riduzione numero di pesi;
3) alternanza livelli di *feature extraction* e *pooling*.

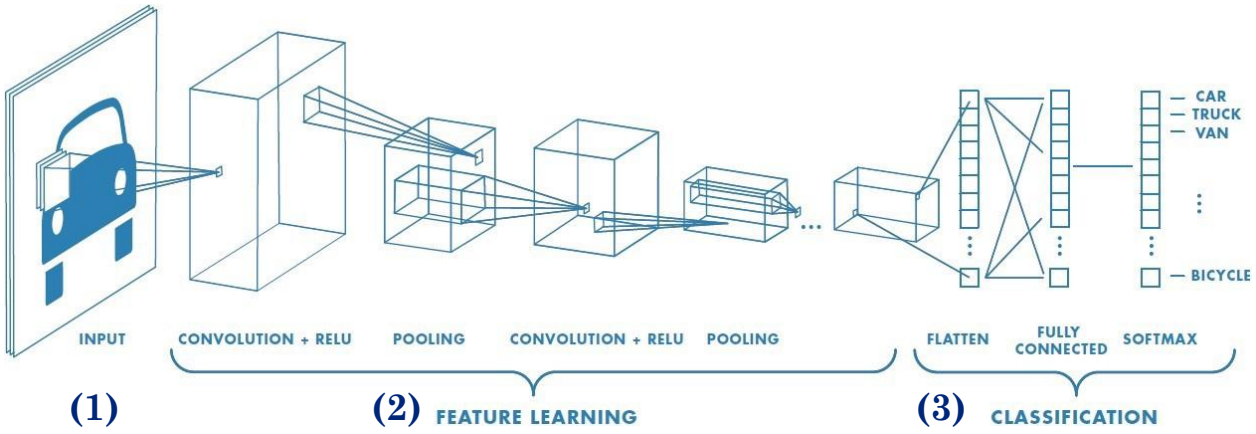


Esplicitamente progettate per processare immagini, per le quali elaborazione locale, pesi condivisi, e pooling non solo semplificano il modello, ma lo rendono più efficace rispetto a modelli *fully-connected*.

Convolutional Neural Network (2)

Una CNN è composta da una gerarchia di livelli in cui:

1) il livello di *input* è direttamente collegato ai pixel dell'immagine;
2) i livelli intermedi presentano connessioni locali e pesi condivisi utili per addestrare la rete ad individuare feature discriminanti;
3) gli ultimi livelli sono generalmente fully-connected e operano come un classificatore.



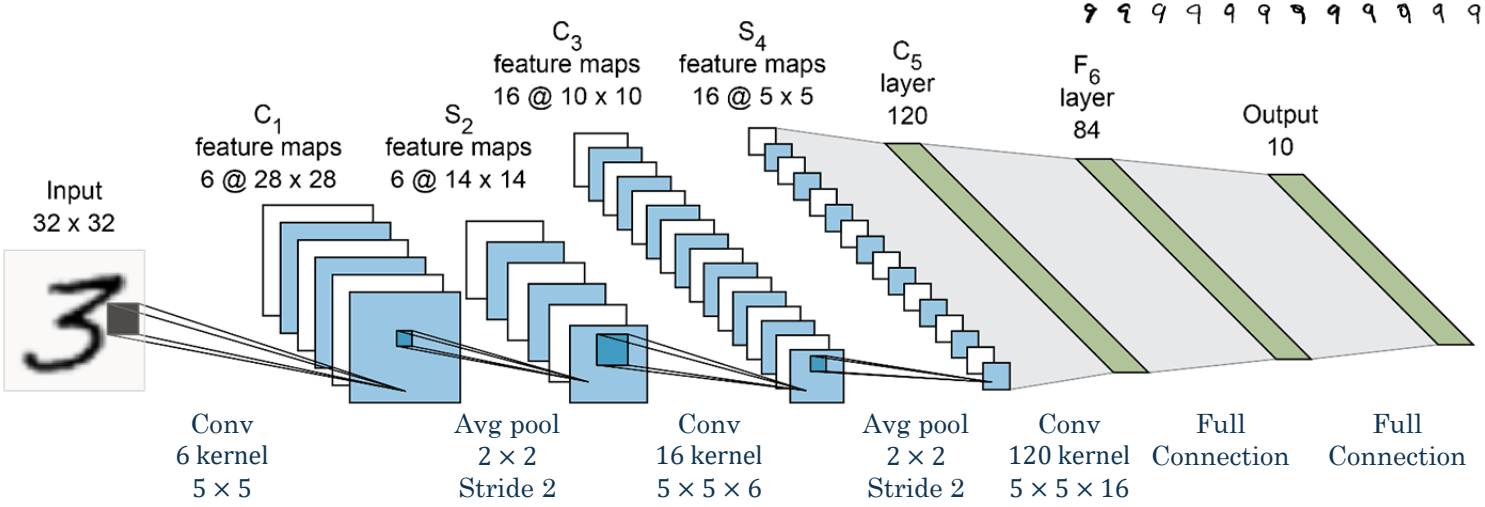
Le connessioni locali e condivise fanno sì che i neuroni processino nello stesso modo porzioni diverse dell'immagine.

LeNet-5

Nel 1998, Lecun, Bottou, Bengio e Haffner introdussero una CNN per classificare cifre scritte a mano. Il modello proposto, chiamato LeNet-5, è composto da:

- tre convolutional layer (C1, C3 e C5);
- due livelli di average pooling (S2 e S4);
- due livelli fully-connected (F6 e Output).





Y. Lecun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," in *proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, November 1998.

LeNet-5 (2)

- E' la prima rete che utilizza convolutional layer e livelli di pooling (che approfondiremo più avanti) per l'estrazione di feature spaziali.
- Per introdurre non-linearità nella rete utilizza la funzione *tanh* nei layer C_1, C_3, C_5 e F_6.
- *Mini-batch Gradient Descent* (SGD - che approfondiremo più avanti) come ottimizzatore durante l'addestramento.
- Circa 60K parametri.
- E' stato utilizzato il database MNIST contenente 70K immagini (60K per il training e 10K per il test) di caratteri scritti a mano 32 × 32 suddivise in 10 classi.
- LeNet-5 ha raggiunto un'accuratezza del 99% sul dataset di test.

LeNet-5 (3)

Contributi

- Utilizzo di convolutional e pooling layer per l'estrazione di feature spaziali.
- LeNet-5 rappresenta il "modello" base per ogni CNN: convolutional layer e livelli di pooling intervallati per la parte di estrazione delle feature e uno o più livelli fully-connected per la parte di classificazione.

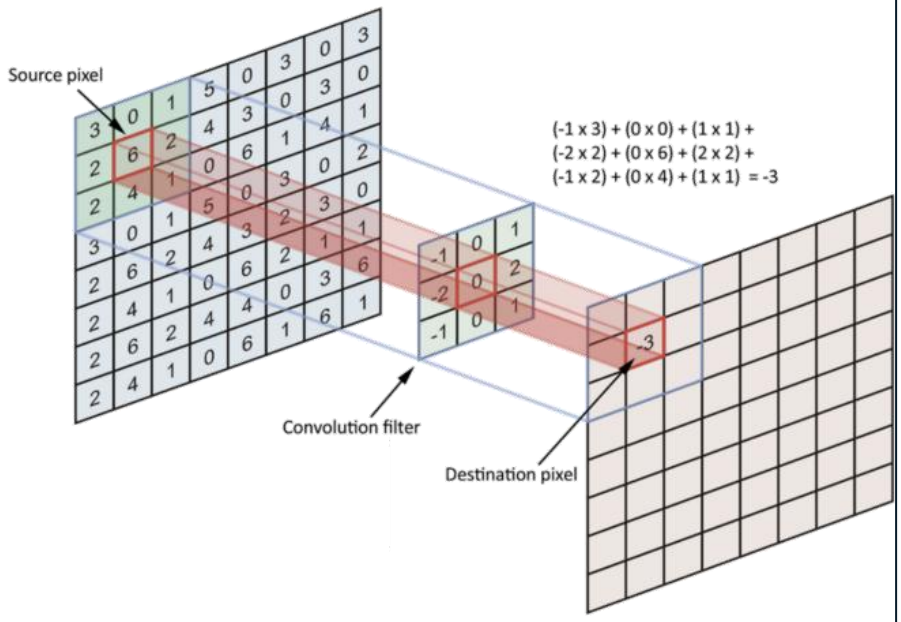
Svantaggi

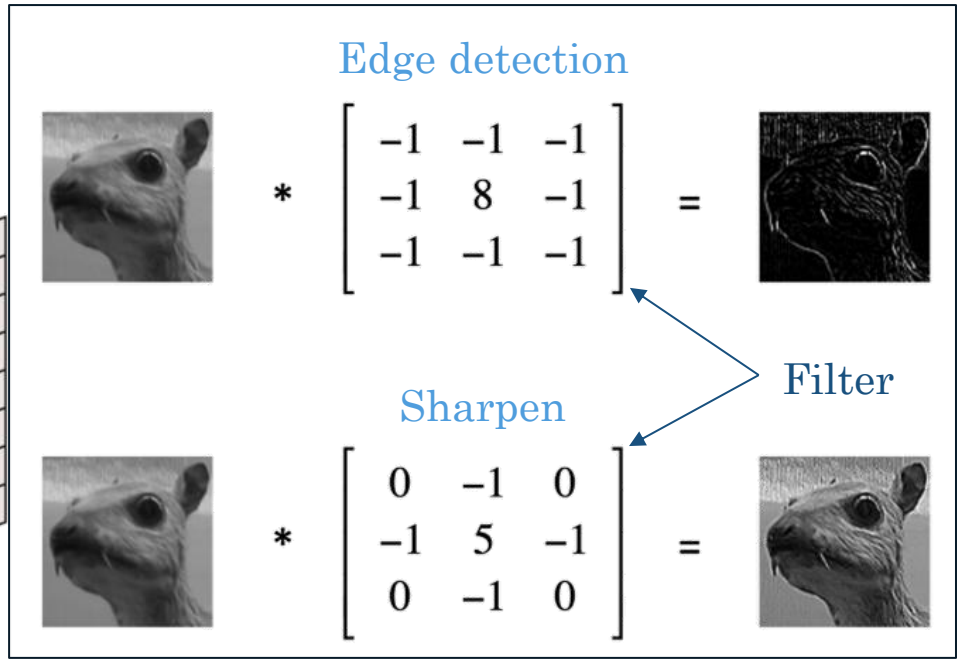
- Utilizzabile su immagini in scala di grigi (1 canale) di piccole dimensioni (32×32).
- Numero di classi limitato (10).

Convolution

La convoluzione è una delle più importanti operazioni di *image processing* attraverso la quale si applicano filtri digitali.

Un filtro digitale è fatto scorrere sulle diverse posizioni di input; per ogni posizione viene generato un valore di output, eseguendo il prodotto scalare di Frobenius tra il filtro e la porzione dell'input coperta.

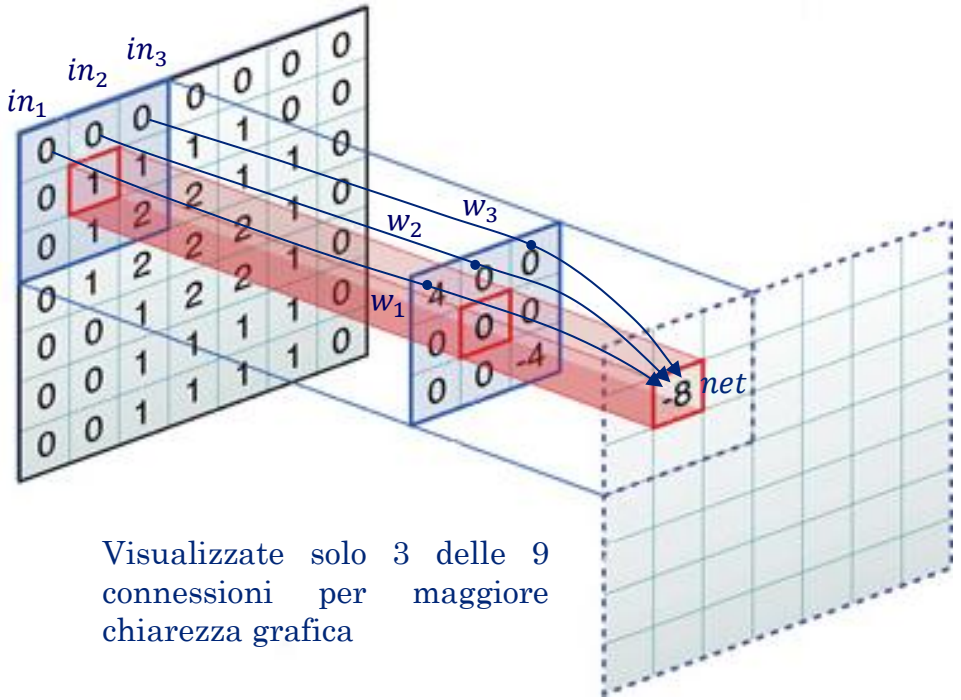




Convolution (2)

Se consideriamo i pixel come neuroni, le immagini di input e di output come livelli successivi di una rete, i pesi del filtro come pesi delle connessioni w, notiamo che un neurone esegue di fatto una convoluzione.

$$net = \sum_{i=1\ldots9} in_i \cdot w_i$$



Visualizzate solo 3 delle 9 connessioni per maggiore chiarezza grafica

Convolution (3)

In una CNN, l'obbiettivo dell'operazione di convoluzione è di estrarre delle feature dal volume di input mantenendo la relazione spaziale tra i neuroni (o pixel dell'immagine).

L'input è composto da un volume 3D (es. l'immagine di input) di dimensione $W_I \times H_I \times C$ e da un insieme di K filtri (o *kernel*) ognuno di dimensione $F \times F \times C$.

L'output è un volume 3D di dimensione $W_O \times H_O \times K$ composto da K *feature map* di dimensione $W_O \times H_O$.

Esempio:

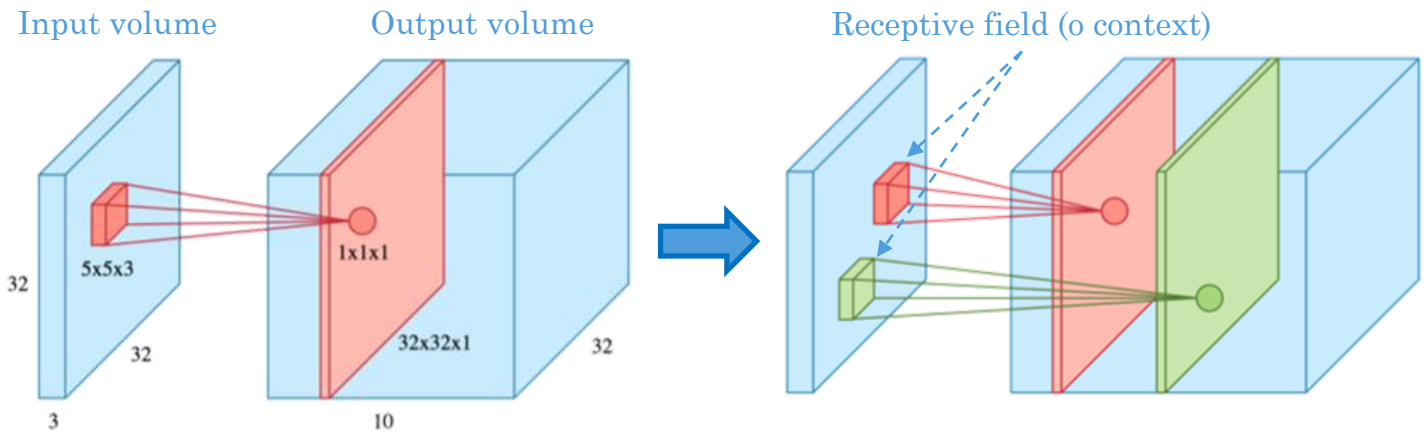
$W_I = H_I = 32$

$C = 3$

$K = 10$

$F = 5$

$W_O = H_O = 32$



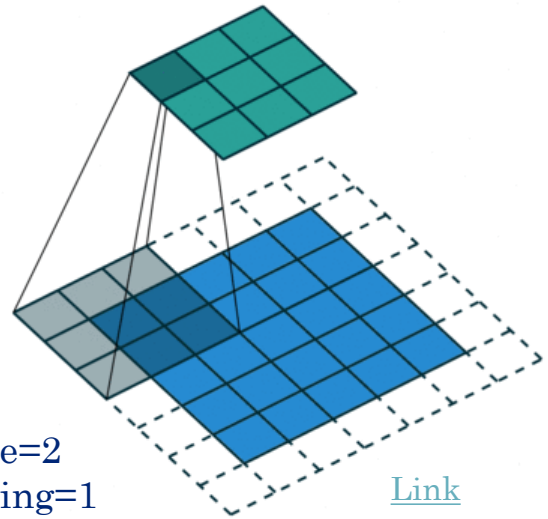
Convolution (4)

Le connessioni locali e condivise fanno sì che porzioni diverse dell'immagine (volume di input) vengano processate nello stesso modo. Si tratta di un comportamento desiderato, in quando regioni diverse del campo visivo contengono lo stesso tipo di informazioni (bordi, spigoli, porzioni di oggetti, ecc.).

Quando un filtro 3D viene fatto scorrere sul volume di input, invece di spostarsi con passi unitari (di 1 neurone) si può utilizzare un passo (o *Stride*) maggiore. Questa operazione riduce la dimensione delle feature map nel volume di output e conseguentemente il numero di connessioni.

Ulteriore possibilità (per regolare la dimensione delle feature map) è quella di aggiungere un bordo (valori zero) al volume di input. Con il parametro *Padding* si denota lo spessore (in pixel) del bordo.

$$W_{out} = \frac{(W_{in} - F + 2 \cdot Padding)}{Stride} + 1$$



ES:
Stride=2
Padding=1

Link

Convolution (5)

Esempio

Stride=1
Padding=0

Input

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

Kernel

0	1	0
0	1	0
0	1	0

Output

18	21	24
33	36	39
48	51	54

$$W_{out} = \frac{5 - 3 + 2 \cdot 0}{1} + 1 = 3$$

Stride=2
Padding=0

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

18	24
48	54

$$W_{out} = \frac{5 - 3 + 2 \cdot 0}{2} + 1 = 2$$

Stride=2
Padding=1

0	0	0	0	0	0	0
0	0	1	2	3	4	0
0	5	6	7	8	9	0
0	10	11	12	13	14	0
0	15	16	17	18	19	0
0	20	21	22	23	24	0
0	0	0	0	0	0	0

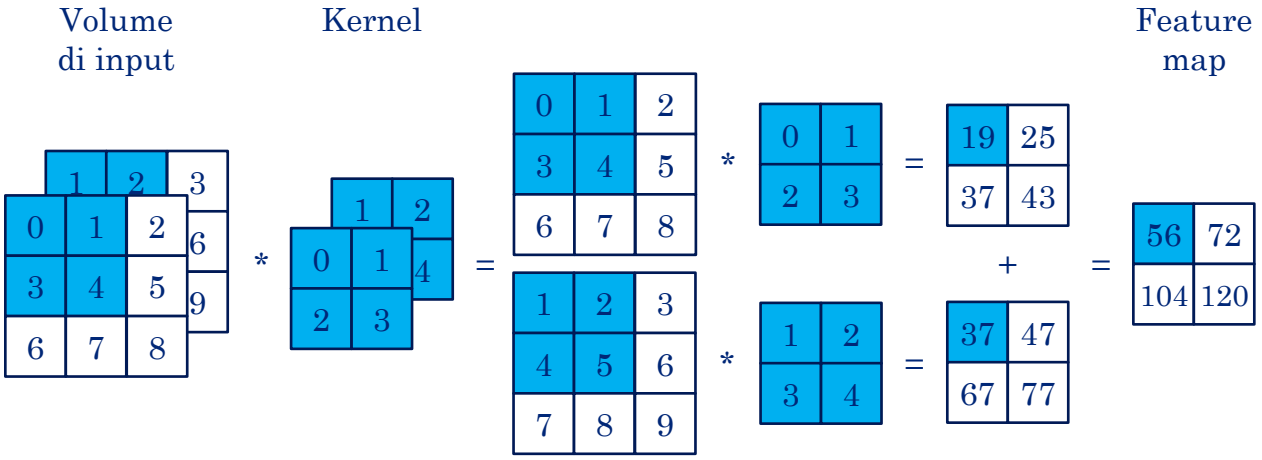
5	9	13
30	36	42
35	39	43

$$W_{out} = \frac{5 - 3 + 2 \cdot 1}{2} + 1 = 3$$

Convolution (6)

I kernel di convoluzione presentano il medesimo numero di canali del volume di input.

Quando il volume di input contiene canali multipli, per ottenere un'unica feature map bidimensionale per ogni kernel, viene eseguita l'operazione di convoluzione tra ogni canale di input e il corrispondente canale del kernel e i risultati ottenuti sono sommati insieme.

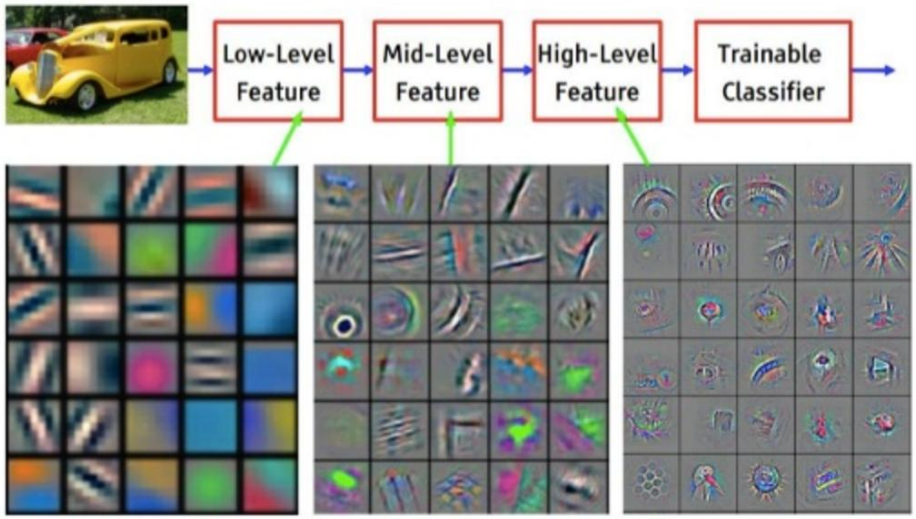


Convolution (7)

IMPORTANTE: una CNN apprende autonomamente i valori dei filtri durante il processo di addestramento.

Solitamente il primo *layer* di convoluzione è responsabile dell'individuazione delle feature di basso livello come bordi, colore, orientamento del gradiente, ecc.

I successivi livelli di convoluzione permettono di distinguere feature sempre di più alto livello dando alla rete una comprensione più dettagliata dell'immagine.

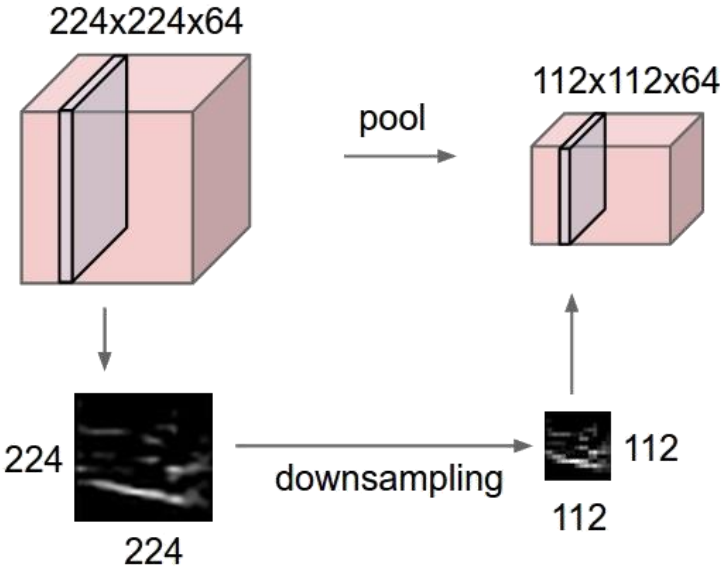


Pooling

L'obbiettivo dell'operazione di *pooling* è di ridurre la dimensione della feature map mantenendo le informazioni più importanti.

In particolare l'operazione di pooling:

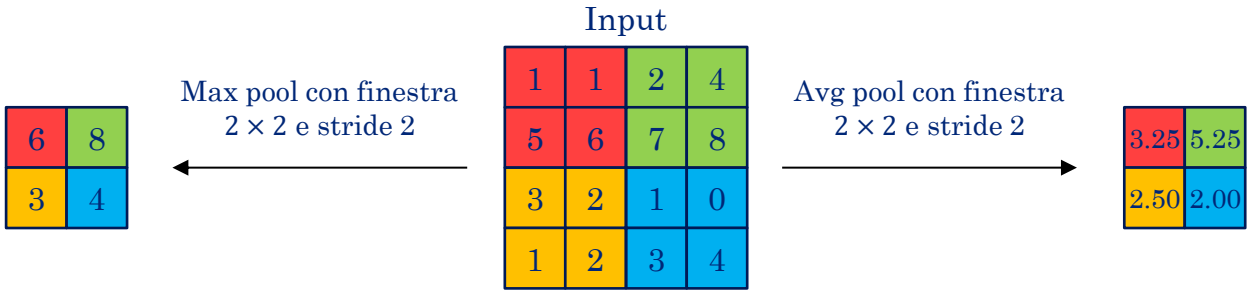
- diminuisce il numero di parametri della rete riducendo il rischio di *overfitting* e la potenza di calcolo necessaria per elaborare i dati;
- rende la rete robusta rispetto a piccole trasformazioni, distorsioni, traslazioni e cambiamenti di scala dell'input.



Pooling (2)

L'operazione di pooling consiste nell'aggregare i valori di input facendo scorrere una finestra di dimensione prefissata e calcolando un singolo valore di ouput per ogni posizione.

L'aggregazione opera (generalmente) nell'ambito di ciascuna feature map, cosicché il numero di feature map nel volume di input e di output è lo stesso. Gli operatori di aggregazione più utilizzati sono la media (Avg) e il massimo (Max).

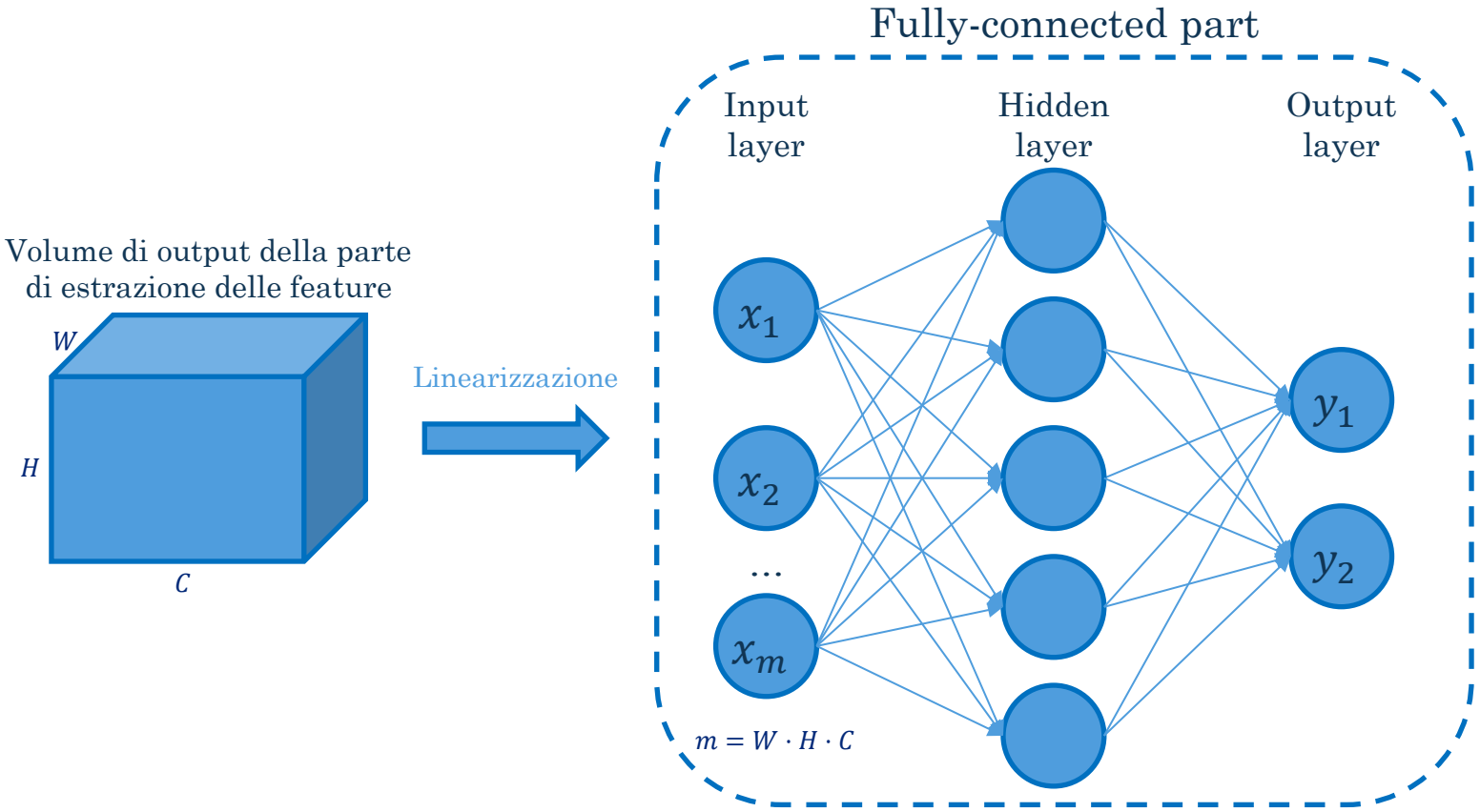


Come per la convoluzione, anche l'operazione di pooling può modificare le dimensioni di output agendo sui parametri di stride e padding.

A differenza di un convolutional layer, in un livello di pooling non vi sono parametri addestrabili.

Fully-connected part

Lo scopo della parte fully-connected è di utilizzare le feature individuate dai livelli precedenti per classificare l'immagine di input in base al training set.

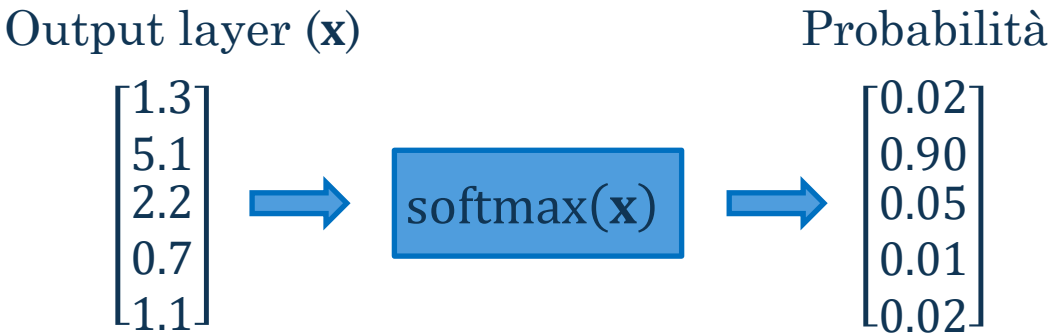


Fully-connected part (2)

L'aggiunta di uno o più livelli fully-connected è anche un modo semplice per apprendere combinazioni non lineari delle feature estratte.

La parte fully-connected è solitamente un MLP che utilizza Softmax come funzione di attivazione del livello di output (assicurando che la somma delle probabilità di output sia 1).

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_k e^{x_k}}$$

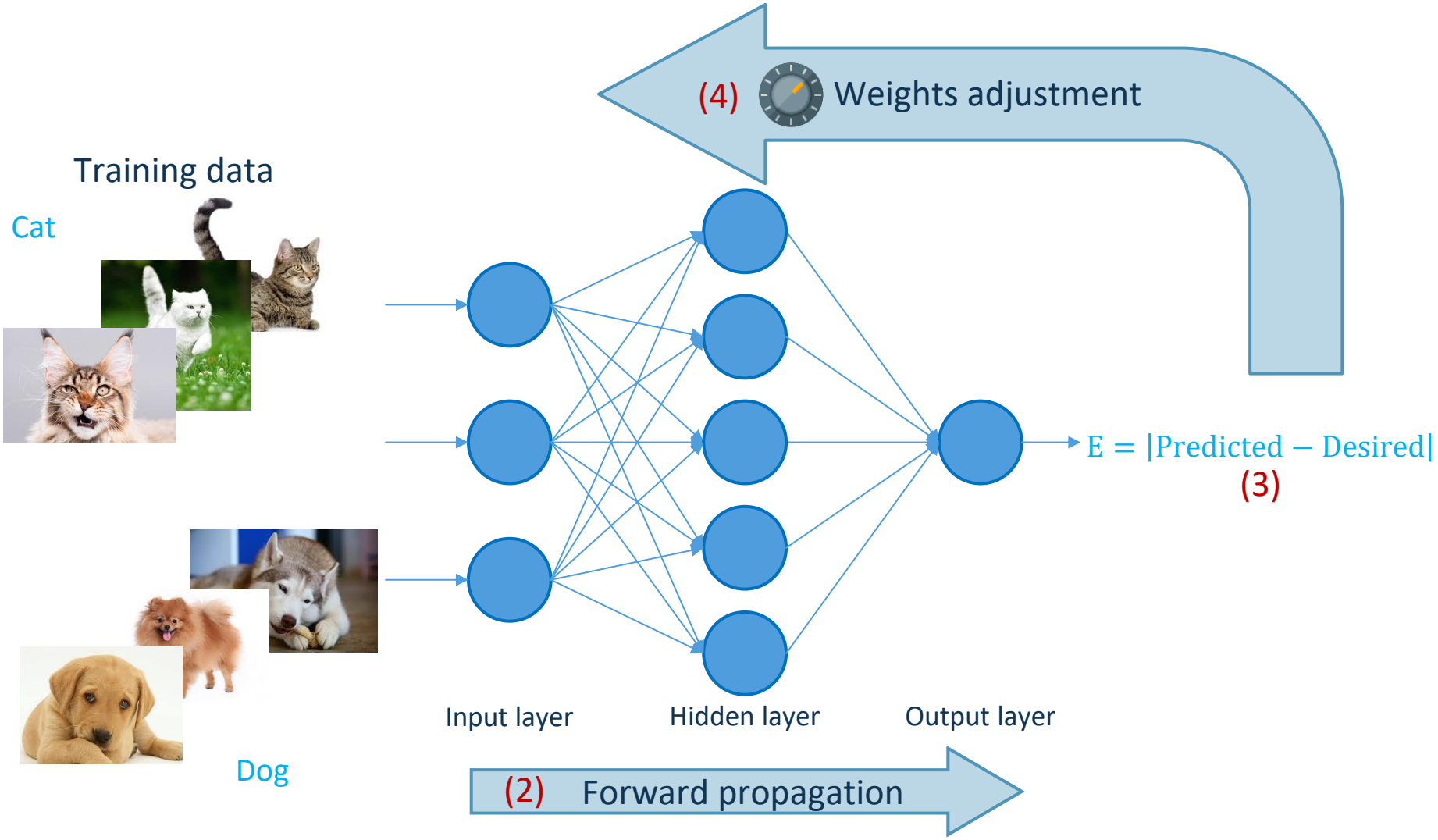


Addestrare una CNN

Il processo di addestramento di una rete neurale può essere riassunto con i seguenti step:

1) inizializzazione casuale dei pesi della rete;
2) un'immagine viene data in input alla rete e tramite la *forward-propagation* si ottengono le probabilità di output per ogni classe;
3) si calcola l'errore del livello di output come scostamento tra l'ouput prodotto e quello desiderato;
4) con la *back-propagation* si calcolano i gradienti dell'errore rispetto a tutti i pesi nella rete e si utilizza la discesa del gradiente per aggiornare tutti i parametri addestrabili della rete così da ridurre al minimo l'errore di output;
5) si ripetono gli step da 2 a 4 per tutte le immagini del training set.

Addestrare una CNN (2)



Mini-batch Gradient Descent

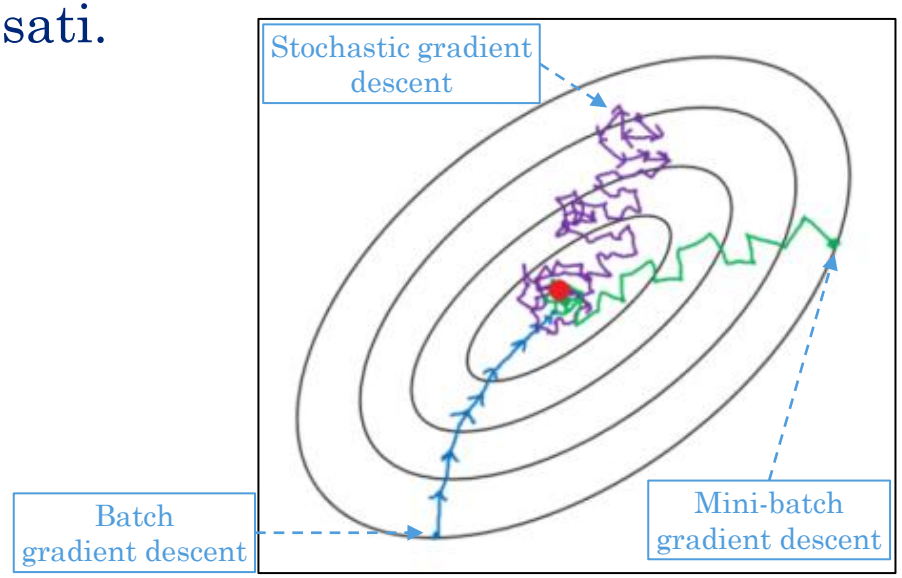
Mini-batch Gradient Descent (o SGD) è l'approccio più utilizzato per l'addestramento di una rete neurale.

- a ogni epoca, gli n pattern del training set sono ordinati in modo casuale e poi suddivisi, considerandoli sequenzialmente, in gruppi (denominati mini-batch) di uguale dimensione ($size_{mb}$).
- i valori del gradiente sono accumulati in variabili temporanee; l'aggiornamento dei pesi avviene solo quando tutti i pattern di un mini-batch sono stati processati.

$size_{mb} = 1 \rightarrow$ Stochastic

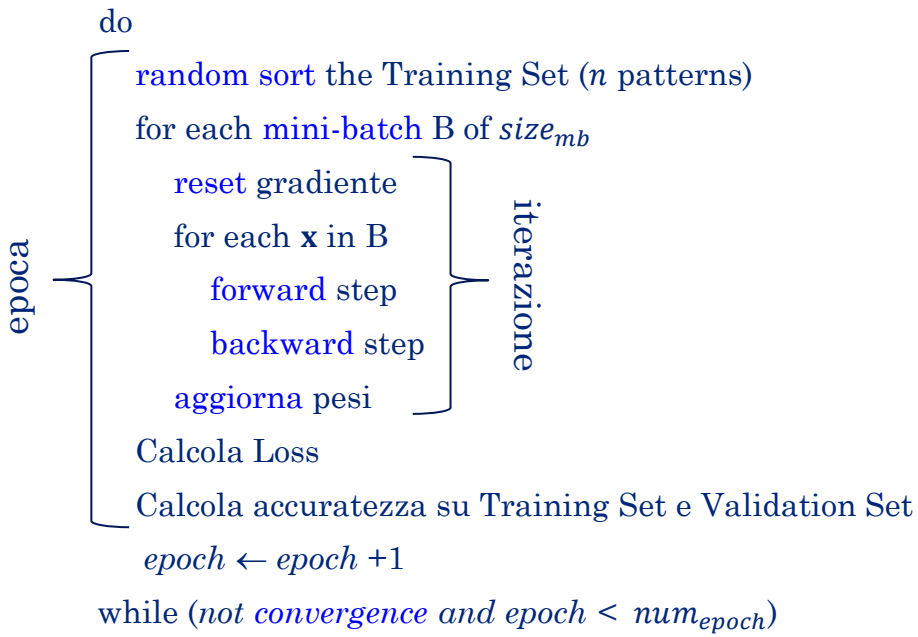
$size_{mb} = n \rightarrow$ Batch

$1 < size_{mb} < n \rightarrow$ Mini-batch



Mini-batch Gradient Descent (2)

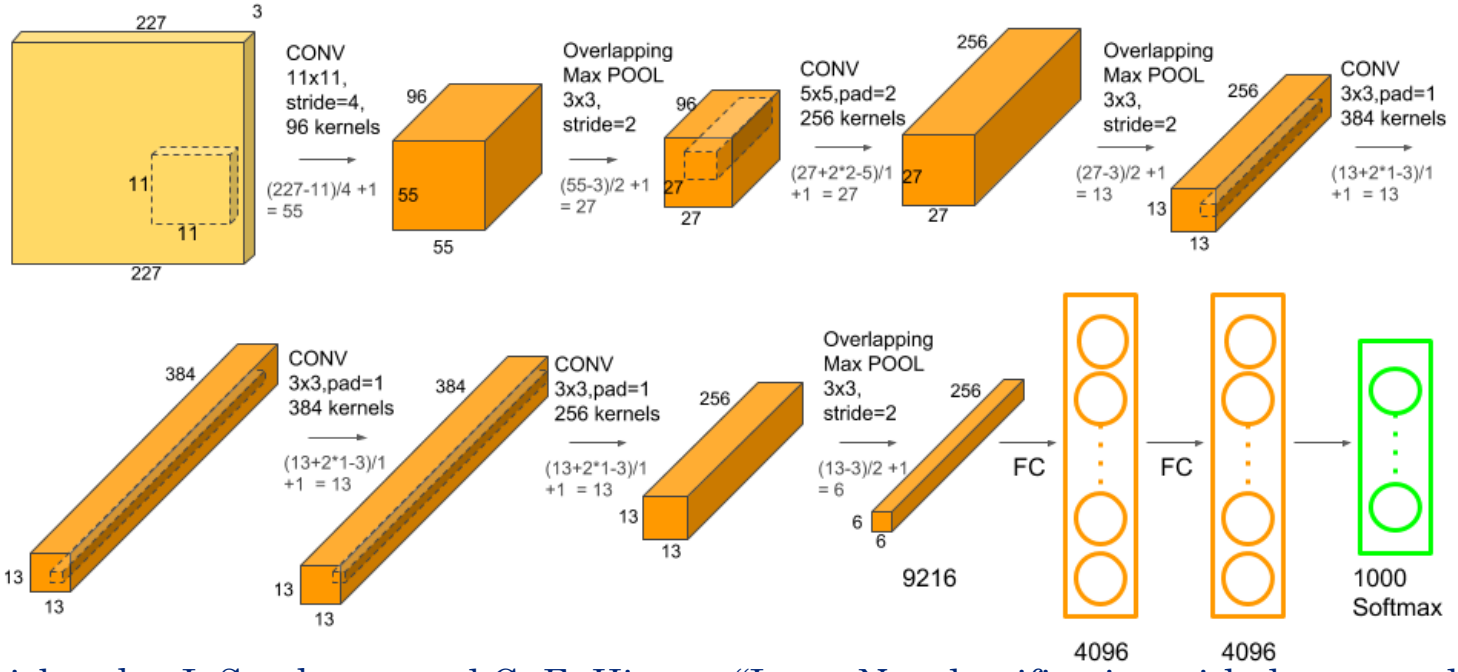
- Epoca: si intende la presentazione (1 volta) alla rete di tutti gli esempi del training set.
- Iterazione: si intende la presentazione (1 volta) degli esempi contenuti in un mini-batch e il conseguente aggiornamento dei pesi.



AlexNet

Nel 2012, Krizhevsky, Sutskever e Hinton proposero AlexNet, una CNN con un'architettura simile a LeNet composta da:

- cinque convolutional layer;
- tre livelli di max pooling;
- tre livelli fully-connected.



A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *proceedings of Advances in Neural Information Processing Systems (NIPS)*, pp. 1106–1114, December 2012.

AlexNet (2)

- È la prima rete a utilizzare *Rectified Linear Unit* (ReLU - che approfondiremo più avanti) per introdurre non-linearità, dopo ogni convolutional e fully-connected layer, aumentando la velocità di apprendimento della rete.
- Utilizza *Local Response Normalization* (LRN) per limitare l'aumento incontrollato dell'output e migliorare la generalizzazione.
- Overlapping max pooling invece di average pooling.
- Per ridurre l'overfitting gli autori hanno utilizzato il *dropout* (che approfondiremo più avanti) nei primi due livelli fully-connected ($p = 0.5$).
- SGD come ottimizzatore durante l'addestramento.
- Addestrata con 1.2M di immagini suddivise in 1000 classi.
- *Data augmentation* (che approfondiremo più avanti) sia in fase di training che di test.
- Con circa 60M di parametri ci sono voluti quasi sei giorni per effettuare l'addestramento in parallelo su due GPU (GTX 580 - 3GB).
- AlexNet ha vinto con largo margine la competizione ImageNet LSVRC-2012 con un error rate di 15.3% (contro il 26.2% del secondo classificato).

AlexNet (3)

Contributi

- ReLU.
- Dropout.
- Data augmentation.
- Addestramento in parallelo su più GPU.

Svantaggi

- Elevato numero di parametri (circa 60M).

Rectified Linear Unit

La funzione di attivazione determina il comportamento del neurone (ovvero il suo output) in funzione del suo livello di eccitazione.

Nelle reti profonde per risolvere il problema del *vanishing gradient* la funzione di attivazione più utilizza è la ReLU (*Rectified Linear Unit*).

$$f(net) = max(0, net)$$

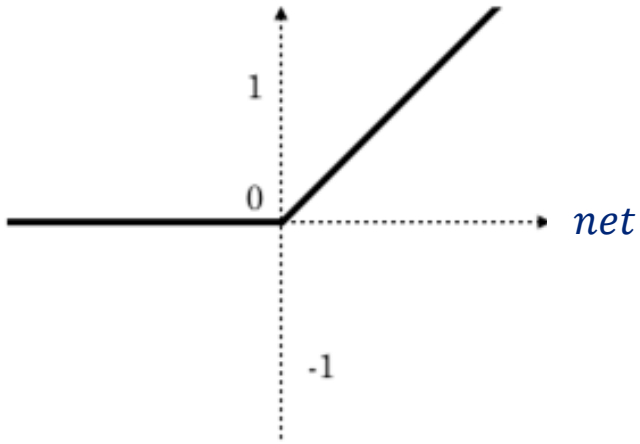
$$f'(net) = \begin{cases} 1 & net > 0 \\ 0 & net < 0 \end{cases}$$

Vantaggi:

- la derivata vale 0 per valori negativi o nulli di *net* e 1 per valori positivi;
- efficiente da calcolare rispetto ad altre funzioni (es. sigmoide o *tanh*).

Svantaggi:

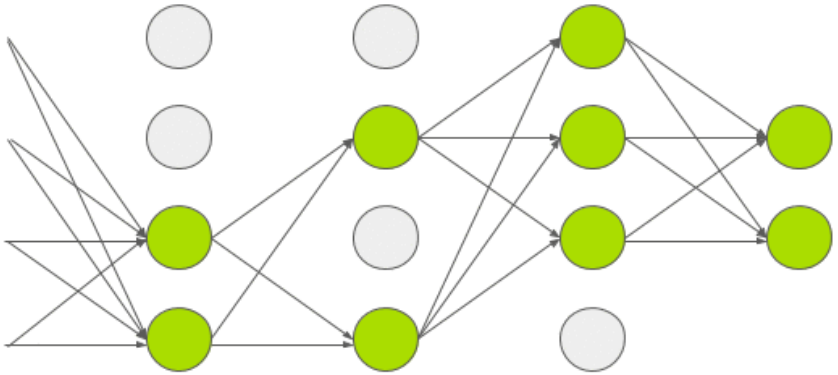
- è una funzione non limitata pertanto potrebbe far "esplodere" il valore di output del neurone;
- per valori negativi di *net* la ReLU restituisce 0 disattivando i corrispondenti neuroni e riducendo la capacità di adattamento della rete.



Dropout

La tecnica chiamata *dropout* introdotta nel 2012 è un metodo di regolarizzazione utile per ridurre l'overfitting causato dall'alto numero di parametri della rete rispetto al numero degli esempi di training.

Durante la fase di training, per ogni immagine di input ogni neurone ha una probabilità p di essere ignorato durante le fasi di *forward* e *backward*. Quindi ogni input passa attraverso una diversa architettura della rete aumentando la robustezza dei parametri appresi.

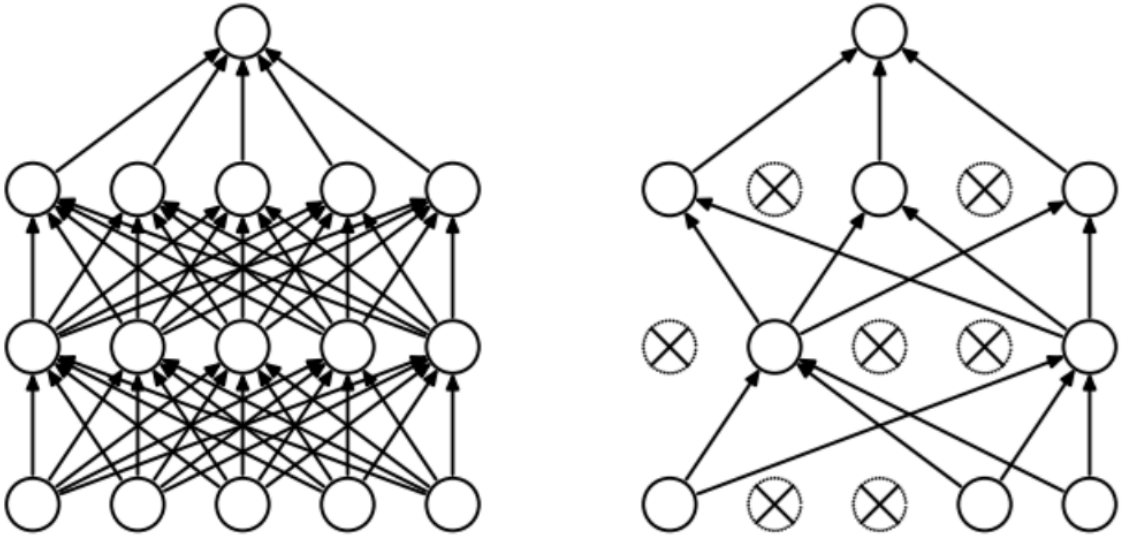


G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," *arXiv 1207.0580*, July 2012.

Dropout (2)

Durante la fase di test viene utilizzata l'intera rete ma l'output viene ridotto di un fattore p per tenere conto dei neuroni ignorati durante la fase di training.

L'utilizzo del dropout aumenta il numero di iterazioni necessarie per convergere ma il tempo necessario per effettuare un'epoca di addestramento si riduce.



(a) Standard Neural Net

(b) After applying dropout.

Data augmentation

La *data augmentation* è una tecnica utilizzata per ridurre l'overfitting e il problema di una disponibilità limitata di dati.

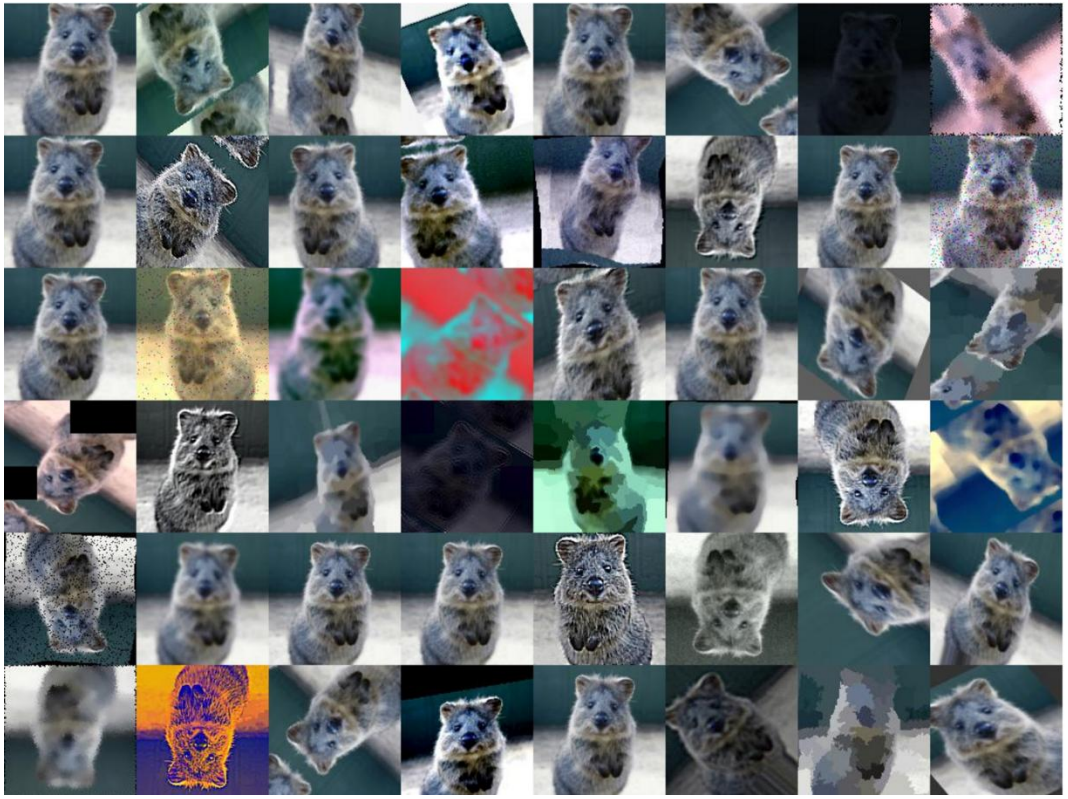
Con il termine data augmentation si identificano una serie di tecniche utilizzate per aumentare la quantità, la qualità e la variabilità del dataset di training aggiungendo copie modificate di dati già presenti o generando nuovi dati sintetici.

Esempi di data augmentation sono:

- traslazione
- rotazione
- riscalatura
- flipping
- stretching
- cropping
- modifica dei colori
- modifica della luminosità
- modifica del contrasto
- aggiunta di rumore
- cancellazioni casuali
- mix di immagini

Data augmentation (2)

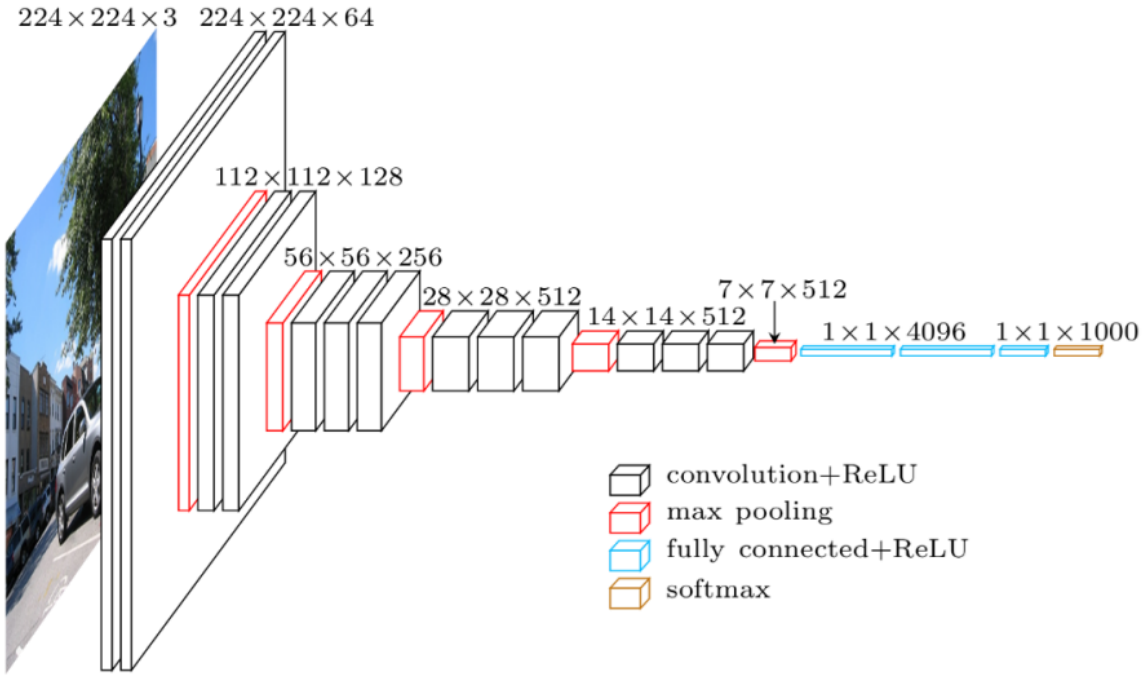




VGGNet

Nel 2014, il *Visual Geometry Group* (VGG) propose VGGNet, una rete con un'architettura simile ad AlexNet composta da:

- 13 convolutional layer (VGG-16);
- cinque livelli di max pooling;
- tre livelli fully-connected.



K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv 1409.1556*, September 2014.

VGGNet (2)

- A differenza di AlexNet, VGGNet applica in cascata kernel a dimensione costante 3×3 riducendo il numero di parametri.
- La normalizzazione LRN non è stata applicata perché aumenta occupazione di memoria e tempi di calcolo senza ottenere un miglioramento delle prestazioni.
- Per ridurre l'overfitting è stato utilizzato *dropout* ($p = 0.5$) nei primi due livelli fully-connected.
- SGD come ottimizzatore durante l'addestramento.
- Varie versioni che condividono il numero di livelli di pooling e fully-connected mentre presentano un numero differente di convolutional layer: VGG-11 (8), VGG-13 (10), VGG-16 (13) e VGG-19 (16).
- Addestramento incrementale partendo da VGG-11 con pesi casuali e aggiungendo nuovi layer per addestrare le versioni successive (mantenendo i pesi delle versioni precedenti).
- Con circa 138M di parametri (VGG-16) ci sono volute circa tre settimane per effettuare l'addestramento in parallelo su quattro GPU (NVIDIA Titan Black).
- VGG-16 è arrivata seconda nella competizione ImageNet LSVRC-2014 con un error rate di 7.3% (dietro a GoogLeNet con un error rate di 6.7%).

VGGNet (3)

Contributi

- Dimostra che la profondità della rete è importante.
- Semplifica la rete per aumentarne la profondità.
- Kernel di piccole dimensioni aiutano a mantenere basso il numero dei parametri.

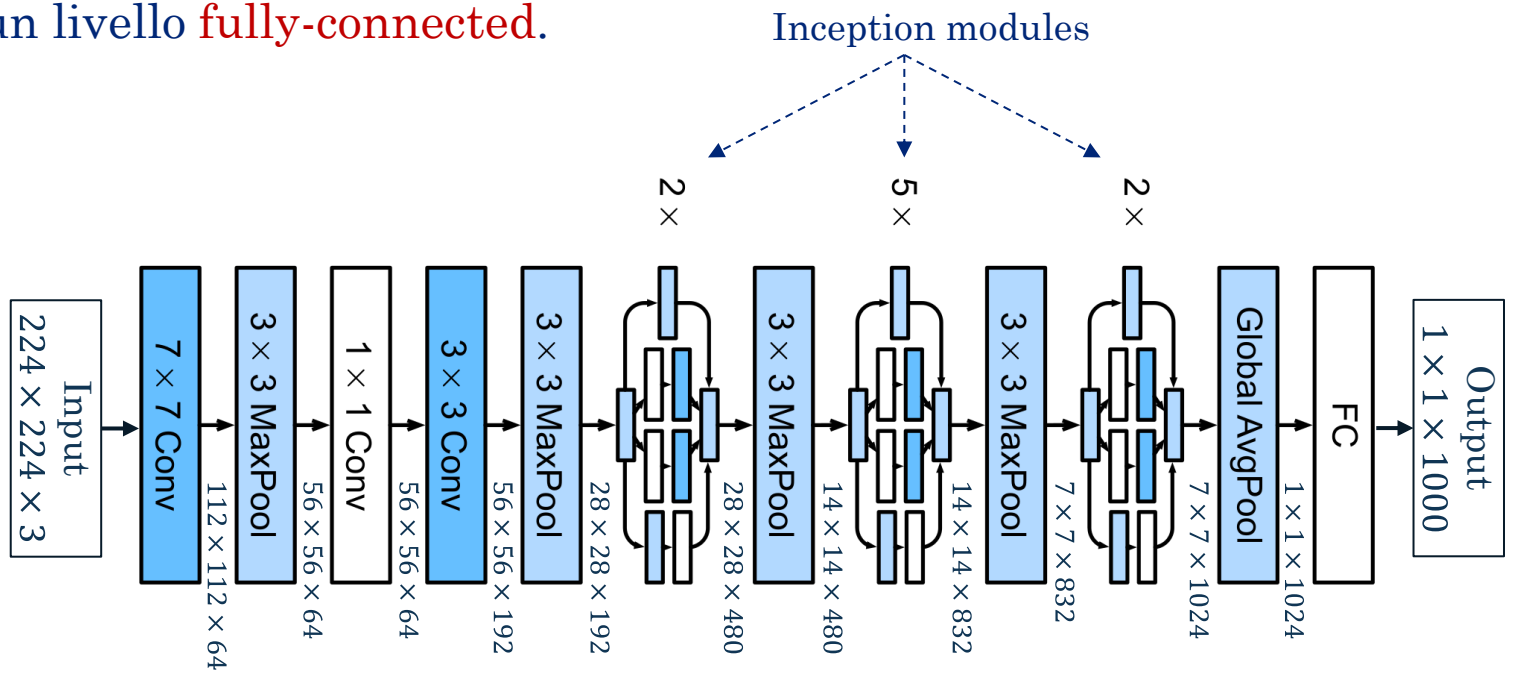
Svantaggi

- Elevato numero di parametri (circa 138M) la maggior parte nei livelli fully-connected (circa 120M).
- Con l'aumentare della profondità della rete la capacità di generalizzazione inizia a degradare (*overfitting* e *vanishing gradient*).

GoogLeNet

Nel 2014, dei ricercatori di Google proposero una nuova architettura GoogLeNet, composta da:

- tre convolutional layer;
- quattro livelli di max pooling;
- nove inception module;
- un livello di global average pooling;
- un livello fully-connected.



C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," *arXiv 1409.4842*, September 2014.

GoogLeNet (2)

- Livelli di convoluzione con kernel 1×1 sono utilizzati per ridurre la complessità computazionale (che approfondiremo più avanti).
- Grazie all'introduzione degli *inception module* (che approfondiremo più avanti) i dettagli dell'immagine possono essere individuati a risoluzioni diverse.
- Un livello di *global average pooling* (che approfondiremo più avanti) è utilizzato per ridurre drasticamente il numero di parametri della parte fully-connected.
- GoogLeNet ha "solo" 7M di parametri (molti meno rispetto ad AlexNet e VGG-16 – 60M e 138M).
- GoogLeNet è arrivata prima nella competizione ImageNet LSVRC-2014 con un error rate di 6.7%.

GoogLeNet (3)

Contributi

- Ha introdotto il concetto di inception module.
- Utilizzo di kernel 1×1 per ridurre la complessità.
- Utilizzo del global average pooling per ridurre il numero di pesi addestrabili.
- Ridotto numero di parametri (circa 7M).

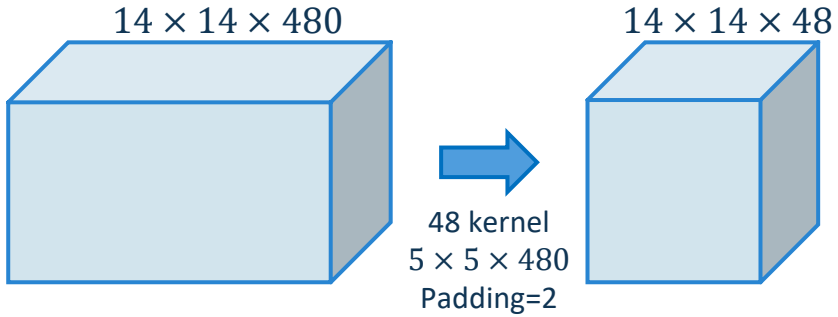
Svantaggi

- Troppo complessa per essere eseguita su dispositivi mobili e/o embedded.

1 × 1 Convolution

Come può essere ridotta la complessità computazionale utilizzando un livello di convoluzione con kernel 1 × 1?

Supponiamo di avere un livello di convoluzione con kernel 5 × 5:

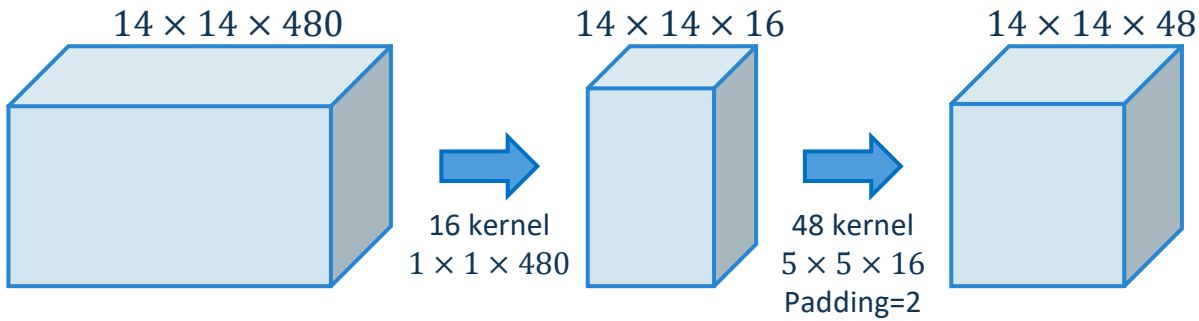


Numero totale di:

- connessioni - $(14 \cdot 14 \cdot 48) \cdot (5 \cdot 5 \cdot 480) \simeq 112.9M$
- pesi - $48 \cdot (5 \cdot 5 \cdot 480) \simeq 576K$

1 × 1 Convolution (2)

Se invece eseguiamo prima un livello di convoluzione con kernel 1×1:



Numero totale di connessioni:

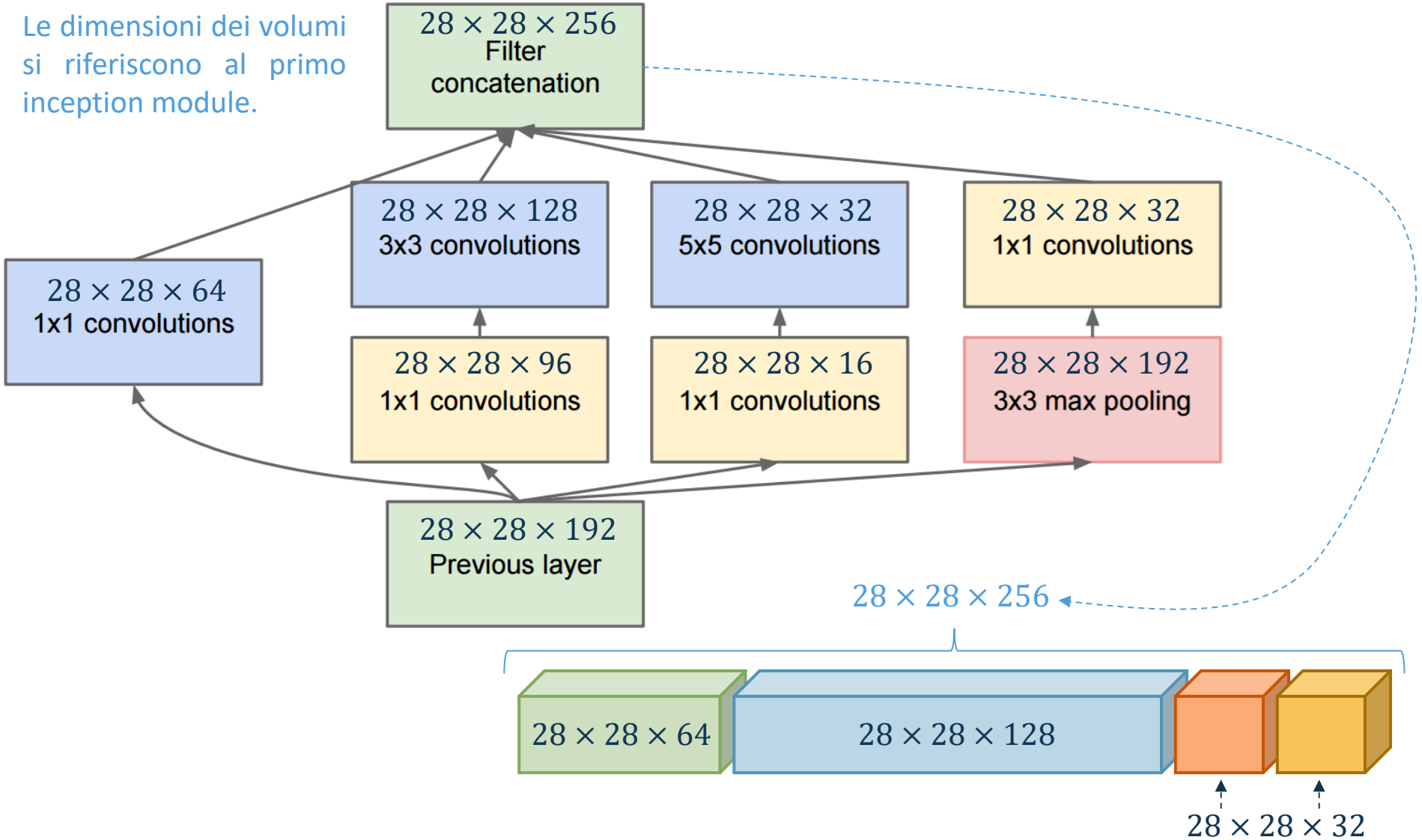
$$\underbrace{(14 \cdot 14 \cdot 16) \cdot (1 \cdot 1 \cdot 480)}_{1 \times 1 \text{ convolution}} + \underbrace{(14 \cdot 14 \cdot 48) \cdot (5 \cdot 5 \cdot 16)}_{5 \times 5 \text{ convolution}} \simeq 5.3\text{M}$$

Numero totale di pesi:

$$\underbrace{16 \cdot (1 \cdot 1 \cdot 480)}_{1 \times 1 \text{ convolution}} + \underbrace{48 \cdot (5 \cdot 5 \cdot 16)}_{5 \times 5 \text{ convolution}} \simeq 26.9\text{K}$$

Inception module

Le dimensioni dei volumi si riferiscono al primo inception module.

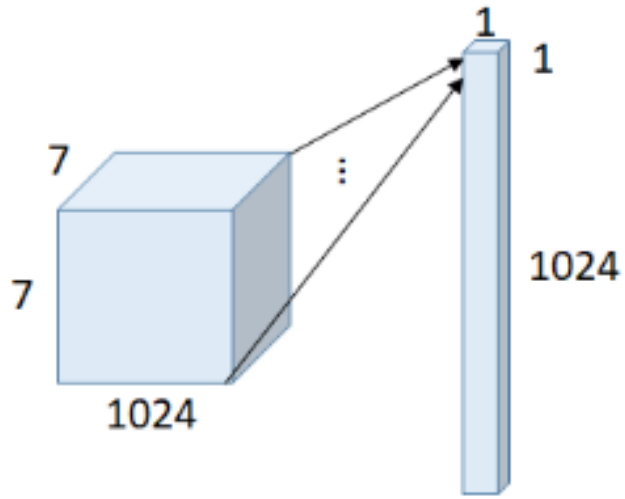


Inception module (2)

- L'inception module è composto da quattro percorsi paralleli:
  - i primi tre utilizzano convolutional layer con kernel di dimensione 1×1, 3×3 e 5×5 per estrarre feature di diverse dimensioni;
  - il secondo e il terzo applicano una preventiva convoluzione 1×1 per ridurre la complessità del modello;
  - il quarto applica un livello di max pooling con finestra 3×3 seguito da un convolutional layer con kernel 1×1 che riduce il numero di canali;
  - tutti utilizzano un padding appropriato per mantenere inalterate le dimensioni spaziali.
- Infine gli output dei quattro percorsi vengono concatenati per restituire un unico volume di output.

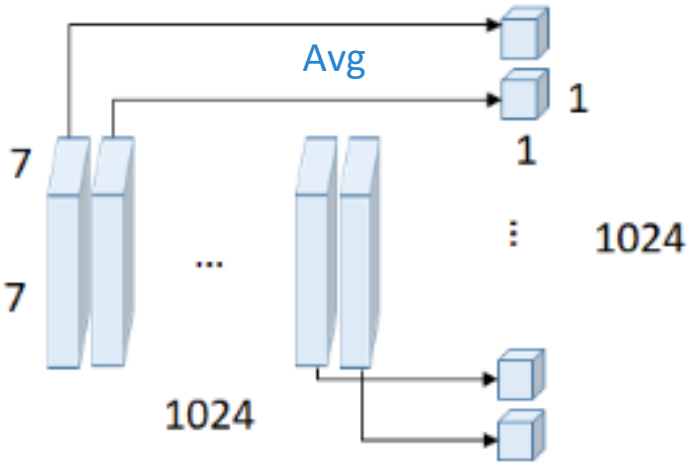
Global average pooling

- Solitamente il volume di output della parte di feature extraction è connesso a un livello fully-connected.
- In GoogLeNet il livello fully-connected è sostituito da un livello di global average pooling per ridurre il numero di pesi (e connessioni).



Fully-connected

weights (connections) = $1024 \cdot (7 \cdot 7 \cdot 1024) \simeq 51.4\text{M}$



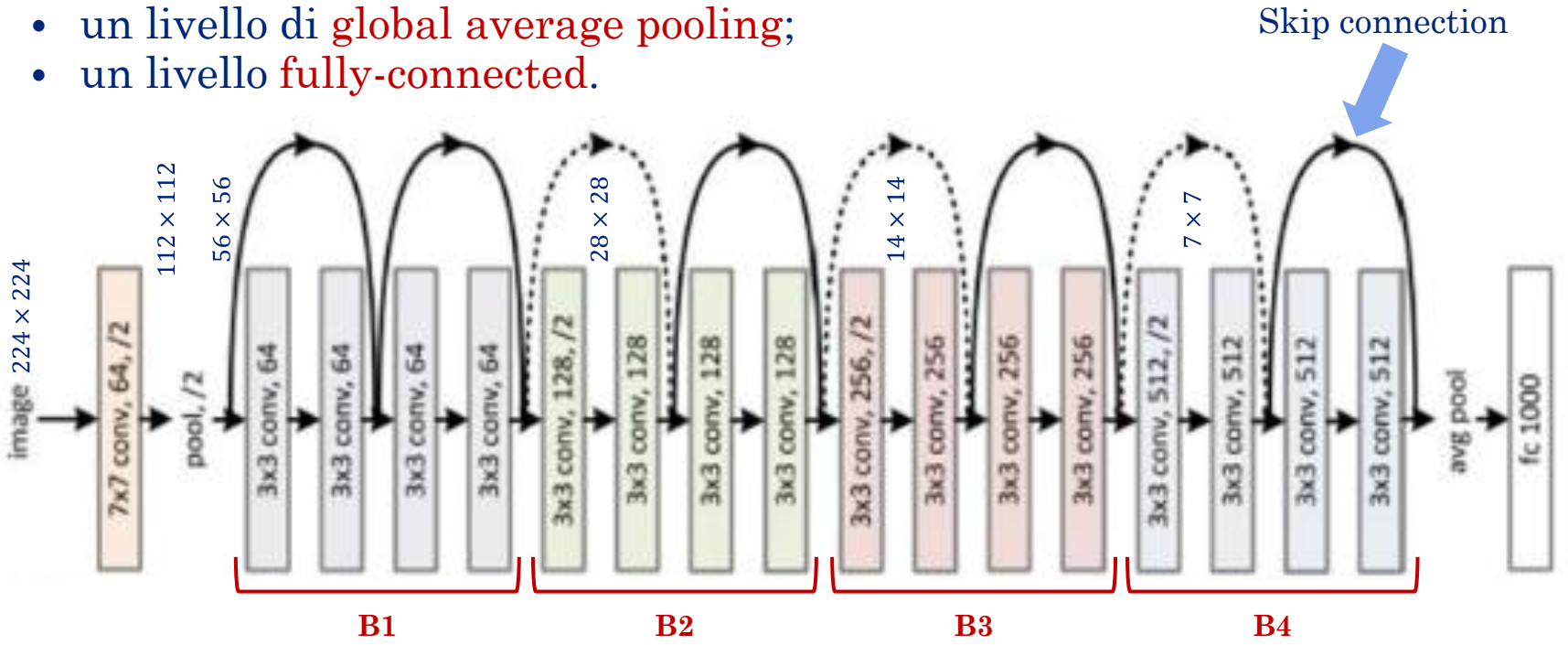
Global average pooling

weights (connections) = 0

ResNet

Nel 2015, He, Zhang, Ren e Sun (Microsoft Research) proposero ResNet (*Residual neural Network*), una rete composta da:

- un convolutional layer;
- un livello di max pooling;
- quattro residual block (B1, B2, B3 e B4);
- un livello di global average pooling;
- un livello fully-connected.



K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *arXiv 1512.03385*, December 2015.

ResNet (2)

- ReLU come funzione di attivazione.
- SGD come ottimizzatore durante l'addestramento.
- Dopo ogni convolutional layer e prima della funzione di attivazione è stata applicata *batch normalization* (che approfondiremo più avanti).
- Per poter addestrare reti molto profonde evitando problemi di degrado delle prestazioni, gli autori hanno utilizzato una sequenza di quattro *residual block* simili tra loro (che approfondiremo più avanti) e le *skip connection*.
- La riduzione delle dimensioni è ottenuta utilizzando uno stride pari a 2 in alcuni convolutional layer invece di impiegare livelli di pooling.

ResNet (3)

- Varie versioni in cui cambia l'architettura e il numero dei sotto-blocchi all'interno di ogni blocco: ResNet-18, ResNet-34, ResNet-50, ResNet-101 e ResNet-152.
- ResNet-50 ha circa 25M di parametri (molti meno rispetto ad AlexNet e VGG-16 – 60M e 138M).
- ResNet è arrivata prima nella competizione ImageNet LSVRC-2015 battendo per la prima volta l'esperto umano (error rate di 5.1%):
  - error rate ResNet-50 – 5.3%
  - error rate ResNet-152 – 4.5%
  - error rate di una combinazione di 6 modelli con diverse profondità – 3.6%

ResNet (4)

Contributi

- Ha reso popolare l'uso delle skip connection (non sono stati i primi ad utilizzarle).
- Ha introdotto il concetto di residual block.
- Permette di progettare CNN con più di 150 livelli senza compromettere la capacità di generalizzazione del modello.
- Tra i primi a utilizzare batch normalization.

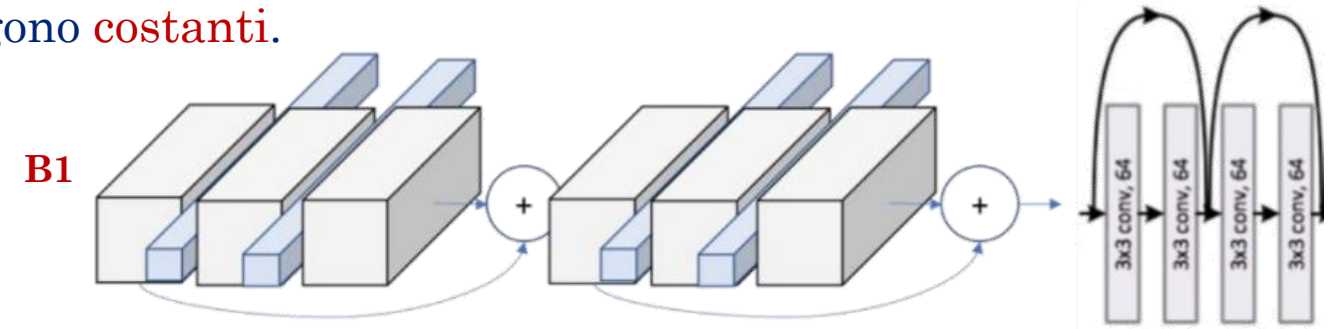
Svantaggi

- Troppo grande e troppo complessa per essere eseguita su dispositivi mobili e/o embedded.

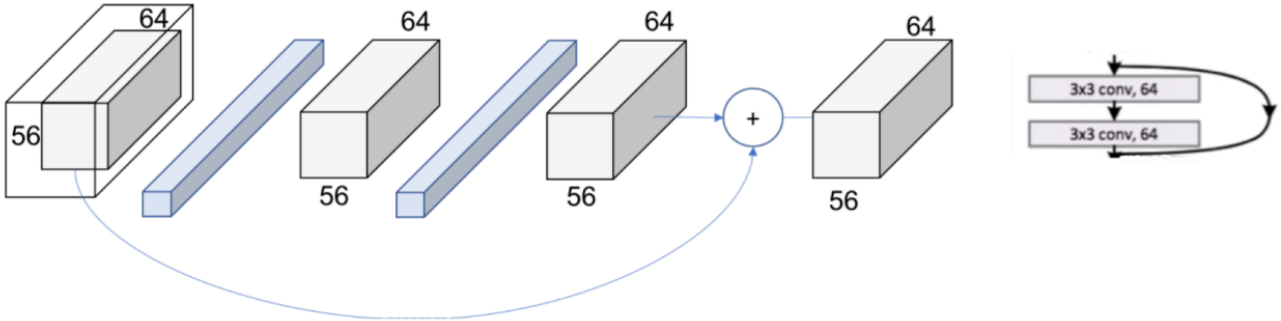
Residual blocks

Ogni blocco è composto da un numero variabile di sotto-blocchi contenenti una serie di convolutional layer con un numero di kernel costante (64, 128, 256 e 512) all'interno del medesimo blocco.

Le dimensioni spaziali delle feature map (*W* e *H*) all'interno di un blocco rimangono costanti.



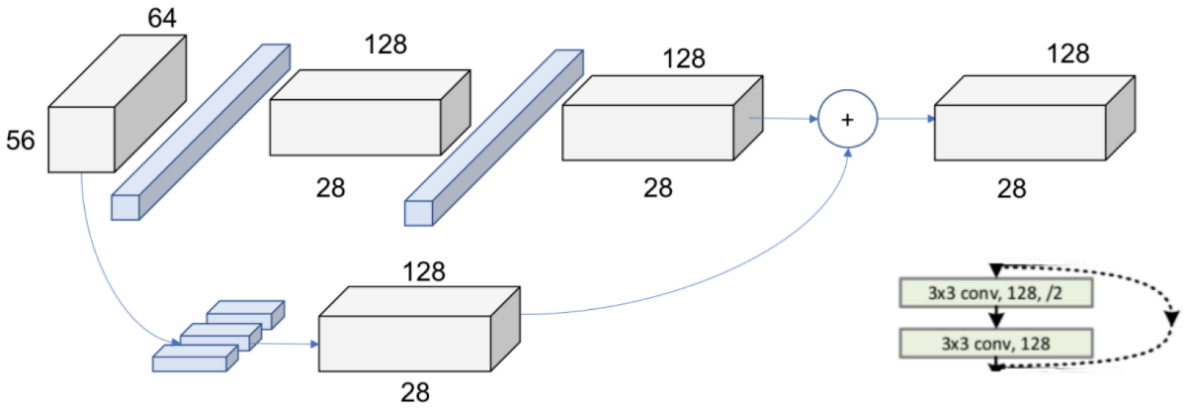
Ogni sotto-blocco contiene al suo interno due convolutional layer e una skip connection per portare l'input del sotto-blocco fino all'output dove viene sommato con l'output del secondo convolutional layer.



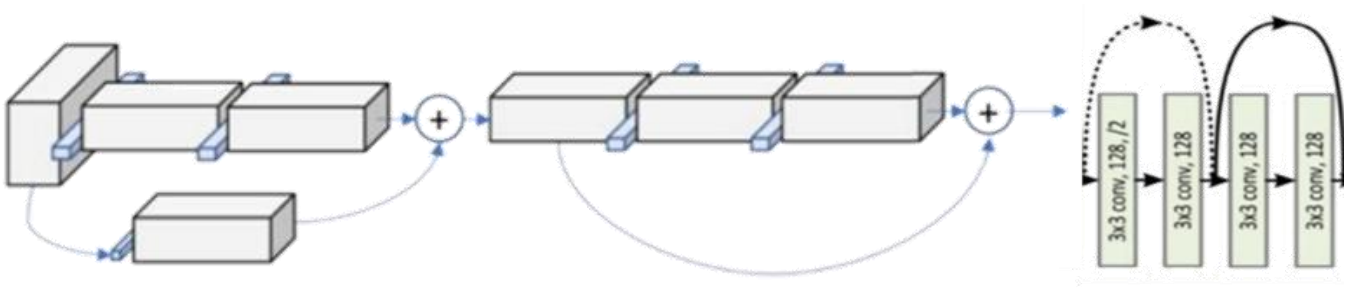
Residual blocks (2)

Alcuni convolutional layer utilizzano uno stride uguale a 2 riducendo le dimensioni spaziali delle feature map.

Nelle skip connection tratteggiate si deve effettua una preventiva riduzione delle dimensioni (tramite un convolutional layer 1×1 con stride 2) prima di poter eseguire la somma delle feature map.



B2



Batch normalization

Batch Normalization (BN) è un livello di normalizzazione addestrabile utilizzato per risolvere il problema chiamato *Internal Covariate Shift* (ICF).

L'ICF è causato dai cambiamenti delle distribuzioni dei neuroni hidden.

Esempio:

Problema binario dove dobbiamo classificare rose e non-rose.



S. Ioffe, and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," *arXiv 1502.03167*, February 2015.

Batch normalization (2)

- I due sottoinsiemi (o mini-batch) si trovano in regioni differenti dello spazio delle feature: questa differenza nelle distribuzioni si chiama *covariate shift*.
- L'addestramento ne risulterà rallentato in quanto per il primo mini-batch la rete tenterà di apprendere una distribuzione molto diversa da quella del mini-batch successivo "rimbalzando" avanti e indietro tra le due distribuzioni fino a quando non convergerà.
- La maniera più semplice di risolvere questo problema per il livello di input è quella di selezionare casualmente i dati al momento della creazione dei mini-batch.
- Il covariate shift può presentarsi anche nei livelli hidden (internal covariate shift) causando un rallentamento nell'addestramento e richiedendo un learning rate piccolo e una buona inizializzazione dei parametri.

Batch normalization (3)

Il problema dell'ICF può essere attenuato normalizzando, per ogni mini-batch, l'output dei neuroni hidden prima di applicare la funzione di attivazione: batch normalization.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

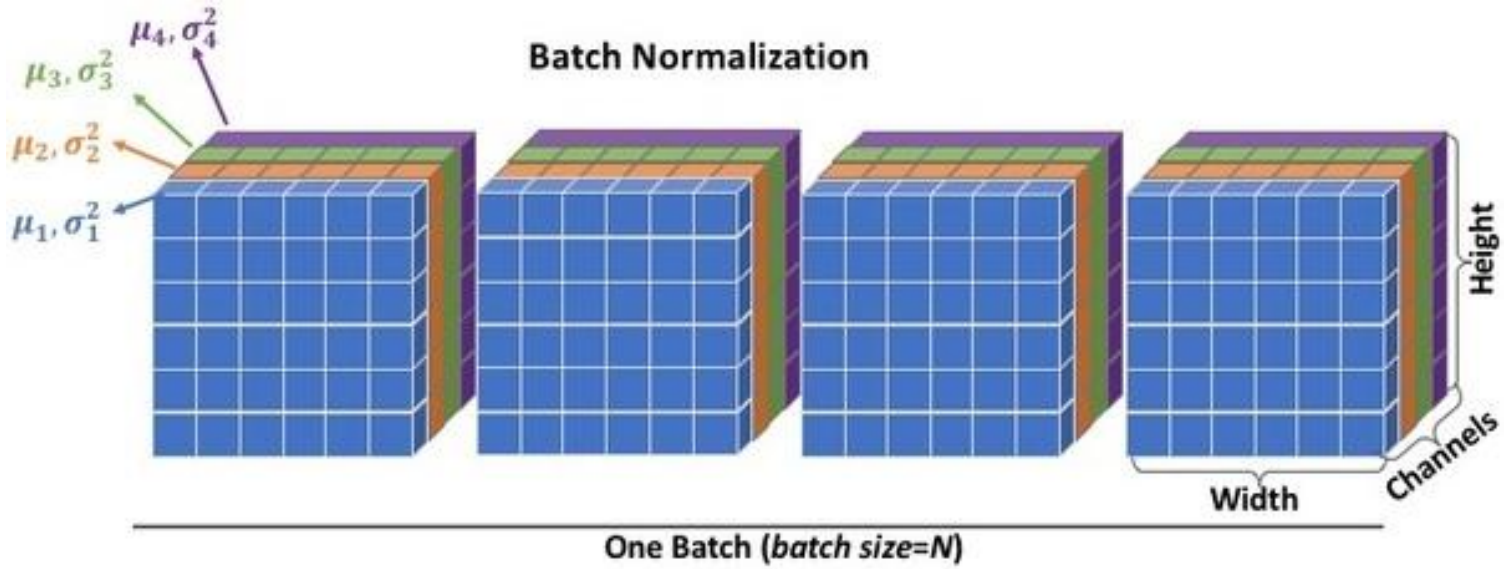
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

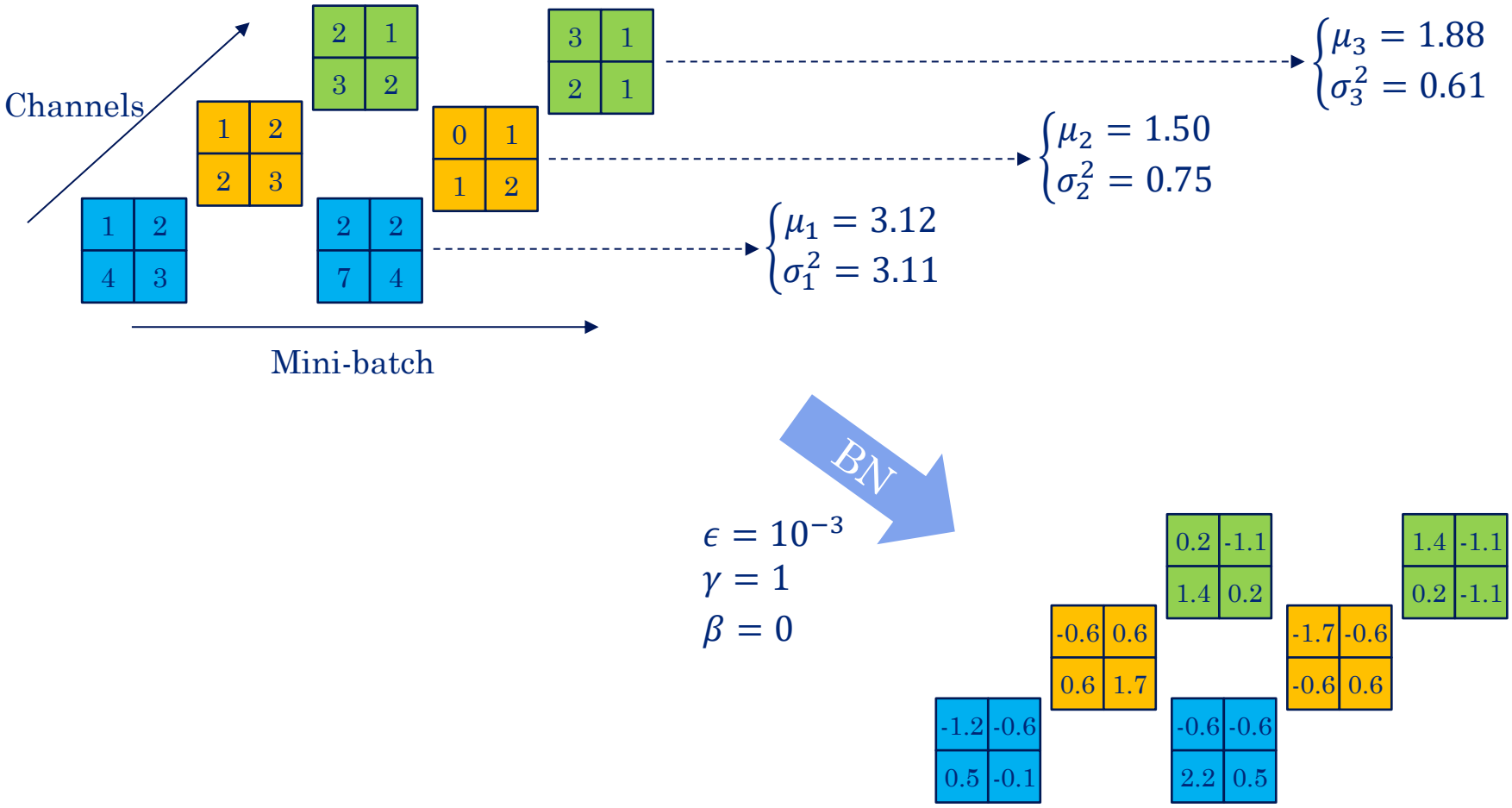
I parametri di riscalatura (γ) e traslazione (β) sono stati introdotti per lasciar decidere alla rete se applicare la normalizzazione o meno ($\gamma = \sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}$ e $\beta = \mu_{\mathcal{B}}$).

Batch normalization (4)



Batch normalization (5)

Esempio



Batch normalization (6)

- Durante la fase di prediction il modello potrebbe essere utilizzato per classificare immagini singole rendendo impossibile il calcolo di μ e σ^2. Alla fine della fase di addestramento vengono calcolate le statistiche di normalizzazione sull'intero dataset di training per utilizzarle nella fase di prediction.
- La dimensione dei mini-batch $size_{mb}$ assume maggiore importanza.
- BN è utilizzato nella maggior parte delle CNN moderne accelerandone considerevolmente la convergenza in fase di addestramento.
- Pur essendo una tecnica vantaggiosa non è dimostrato che riduca effettivamente l'internal covariate shift.

MobileNet

Nel 2017, Google ha proposto MobileNet, una rete progettata per funzionare su dispositivi mobile e/o embedded, composta da:

- 27 convolutional layer;
- un livello di global average pooling;
- un livello fully-connected.

Google Vision Kit



Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
$5\times$ Conv dw / s1	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
$5\times$ Conv / s1	$1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W., T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv 1704.04861*, April 2017.

MobileNet (2)

- Livelli di *depthwise separable convolution* (che approfondiremo più avanti) sono utilizzati per sostituire i classici convolutional layer riducendo drasticamente il costo computazionale.
- Dopo ogni livello e prima della funzione di attivazione è stato applicato batch normalization.
- ReLU come funzione di attivazione ad eccezione dell'ultimo livello fully-connected in cui viene utilizzato Softmax.
- Riduzione delle dimensioni spaziali tramite stride in alcuni livelli di convoluzione.
- RMSProp come ottimizzatore durante l'addestramento.

MobileNet (3)

- Introdotti due iperparametri per poter ridurre ulteriormente il modello e il costo computazionale della rete: *width multiplier* α e *resolution multiplier* ρ.
- Data augmentation minima durante la fase di training visto che modelli di piccole dimensioni presentano meno problemi di overfitting.
- MobileNet-224 ha circa 4.2M di parametri (molti meno rispetto ad AlexNet, VGG-16 e ResNet50 – 60M, 138M e 25M).
- MobileNet-224 ha raggiunto su ImageNet un'accuratezza pari all'89.5% (error rate del 10.5%) superando AlexNet pur avendo un numero nettamente inferiore di parametri.

MobileNet (4)

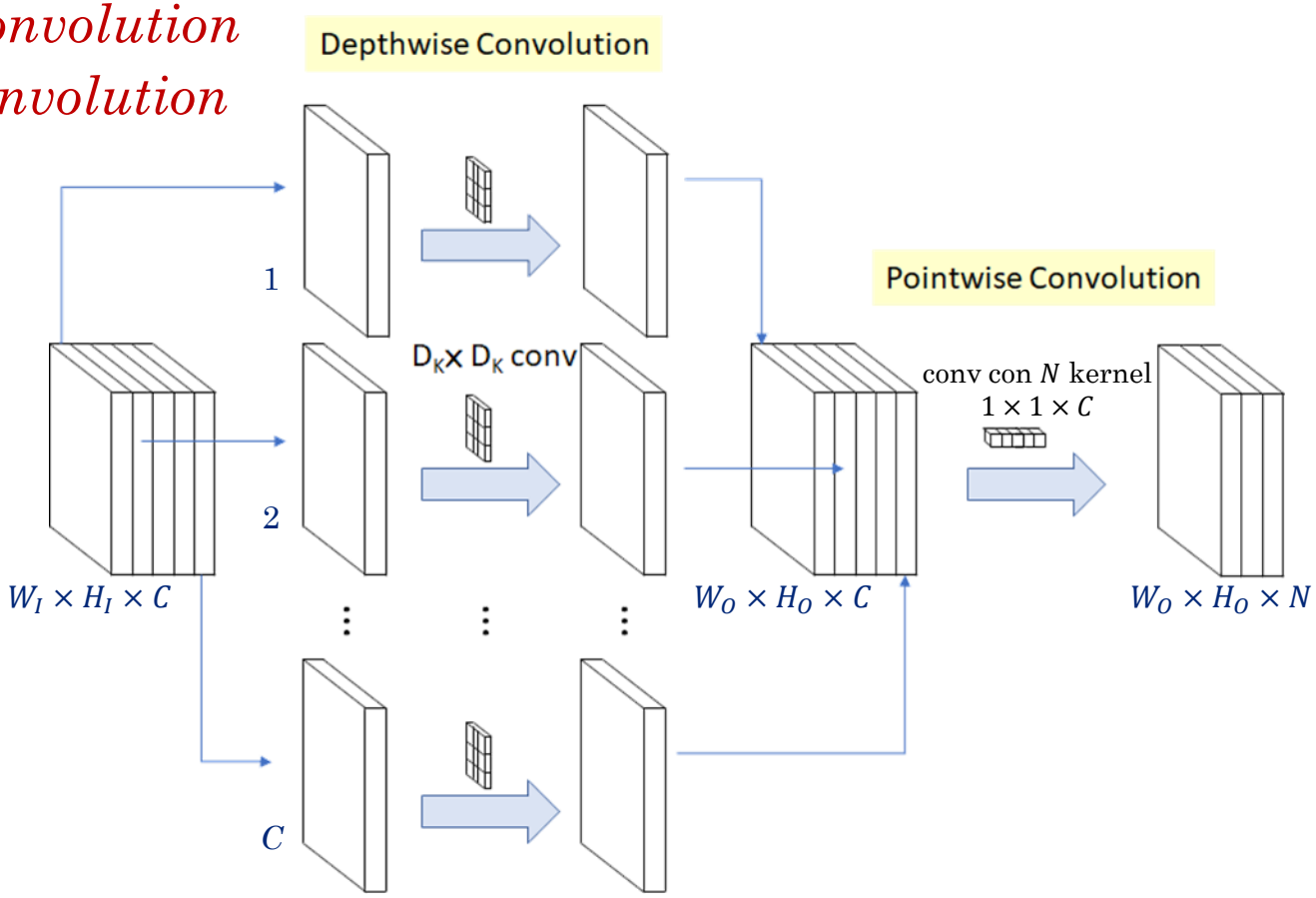
Contributi

- Modello di piccole dimensioni.
- Complessità ridotta.
- Adatta a essere utilizzata su sistemi mobile e embedded.

Depthwise separable convolution

La depthwise separable convolution divide un kernel in due kernel separati ed esegue in sequenza due operazioni di convoluzione:

1. *depthwise convolution*
2. *pointwise convolution*

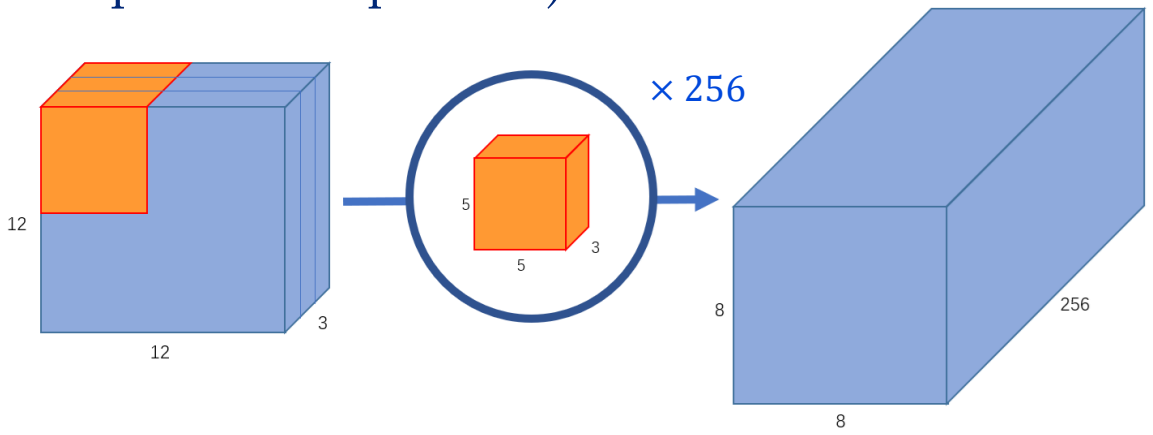


Depthwise separable convolution (2)

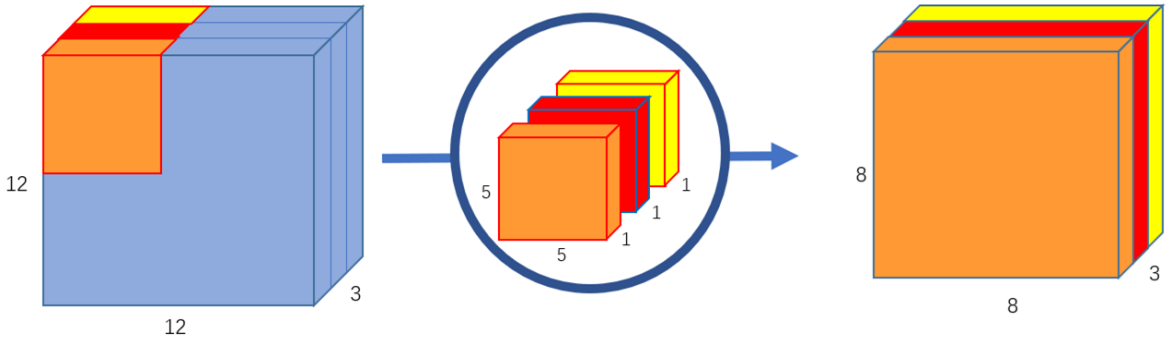
A differenza dell'operazione classica di convoluzione, in cui ogni kernel viene applicato a tutto il volume di input, nella depthwise convolution si hanno tanti kernel quant'è la profondità del volume di input (C) e ogni kernel viene applicato solamente a uno specifico strato dell'input (pertanto ogni kernel avrà profondità pari a 1).

Esempio:

Convoluzione classica

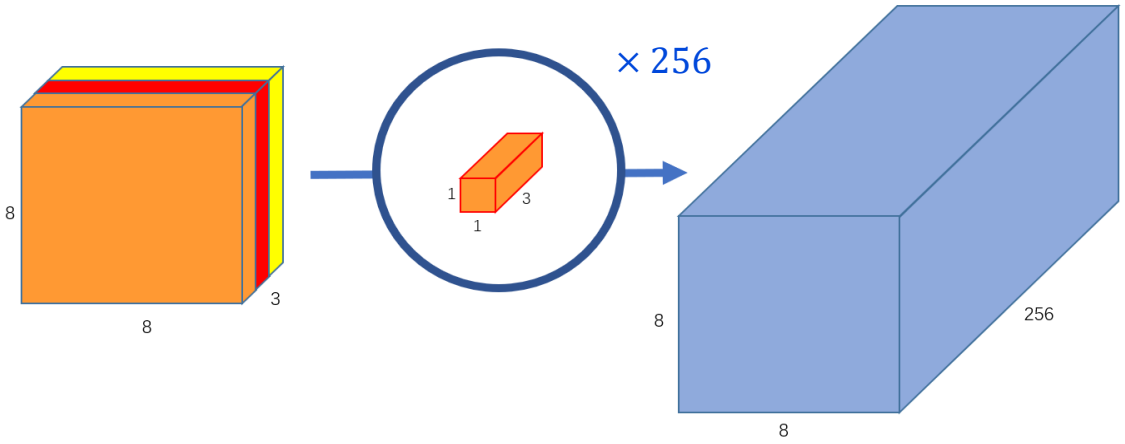


Depthwise convolution



Depthwise separable convolution (3)

La pointwise convolution non è altro che un'operazione di convoluzione classica in cui ogni kernel ha dimensione $1 \times 1 \times C$ (dove C è la profondità del volume di input).



Applicando in sequenza una depthwise convolution e una pointwise convolution è possibile ottenere un volume di output con le stesse dimensioni di quello ottenuto con una convoluzione classica.

Depthwise separable convolution (4)

Quali sono i vantaggi a utilizzare la depthwise separable convolution?

Confrontiamo il numero di moltiplicazioni necessarie per eseguire i due tipi di convoluzione.

Convoluzione classica: $W_O \cdot H_O \cdot N \cdot F \cdot F \cdot C$

Depthwise separable convolution:

$$\underbrace{W_O \cdot H_O \cdot C \cdot F \cdot F \cdot 1}_{Depthwise\ convolution} + \underbrace{W_O \cdot H_O \cdot N \cdot 1 \cdot 1 \cdot C}_{Pointwise\ convolution}$$

Nell'esempio delle slide precedenti:

Convoluzione classica: $8 \cdot 8 \cdot 256 \cdot 5 \cdot 5 \cdot 3 \simeq 1229\text{K}$

Depthwise separable convolution:

$8 \cdot 8 \cdot 3 \cdot 5 \cdot 5 \cdot 1 + 8 \cdot 8 \cdot 256 \cdot 1 \cdot 1 \cdot 3 = 4800 + 49152 \simeq 54\text{K}\ (4.4\%)$

Depthwise separable convolution (5)

Ci sono anche degli svantaggi?

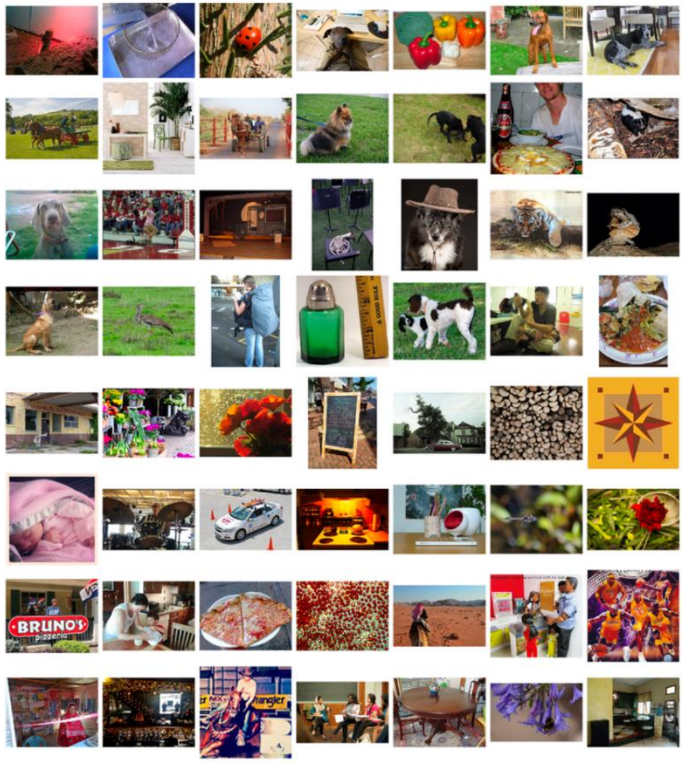
Riducendo il numero di parametri della convoluzione , se la rete è troppo piccola potrebbe non essere in grado di apprendere correttamente durante l'addestramento.

Se usata correttamente è una tecnica che permette di migliorare l'efficienza della rete senza ridurne in maniera significativa l'accuratezza.

Modelli a confronto

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) è una competizione internazionale con cadenza annuale in cui i partecipanti competono per ottenere le migliori performance su diversi problemi di *computer vision* su un sottoinsieme di immagini del database ImageNet.

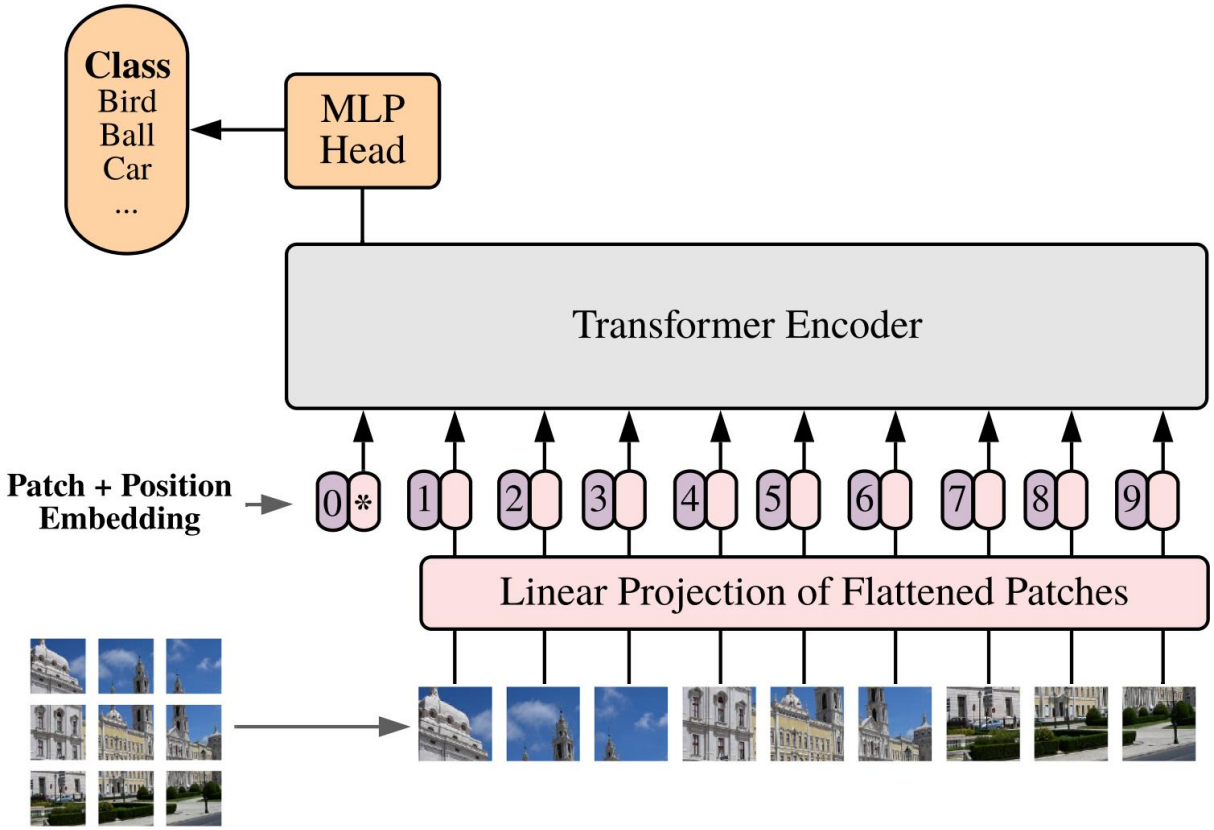
- 1000 classi
- 1.2M immagini di training
- 50K immagini di validation
- 100K immagini di test



Model	# Layer	# Parametri	MAdds	error rate
LeNet-5	5	60K	-	-
AlexNet	8	60M	0.7G	15.3%
VGG-16	16	138M	15.3G	7.3%
GoogLeNet	22	7M	1.6G	6.7%
ResNet-50	50	25M	3.9G	5.3%
DenseNet-121	121	7M	2.8G	-
MobileNet-224	224	4.2M	0.6G	10.5%
SENet	-	-	-	2.3%

ViT

Nel 2020, il team Google Research Group ha proposto ViT (*Vision Transformer*), una rete basata su Transformer.



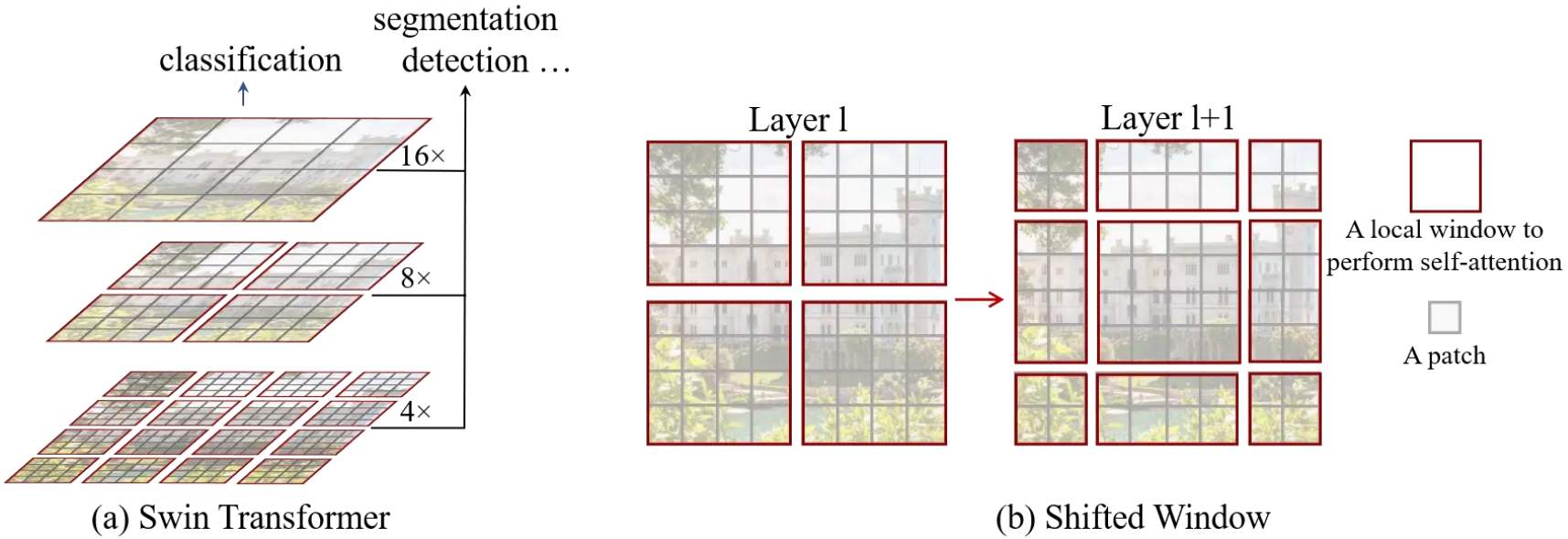
A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *arXiv 2010.11929*, October 2020.

ViT (2)

- È il primo Transformer progettato per svolgere un task di *image recognition*.
- L'immagine viene suddivisa in regioni non sovrapposte di dimensione prefissata (16×16).
- Ogni regione viene linearizzata e integrata con la posizione relativa all'interno dell'immagine originale così che il modello la utilizzi per ricostruire la struttura dell'immagine.
- I vettori così ottenuti vengono dati in input a un Transformer Encoder che codifica le relazioni che intercorrono tra le varie regioni dell'immagine.
- ViT ha raggiunto su ImageNet un'accuratezza pari all'88% (error rate del 12%) mostrando risultati comparabili con le CNN.

Swin

Nel 2021, il team Microsoft Research Asia ha proposto Swin (*Shifted Window*), una rete basata su Transformer gerarchico progettata per affrontare task di visione artificiale.



(a) Swin Transformer

(b) Shifted Window

Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," *arXiv 2103.14030*, March 2021.

Swin (2)

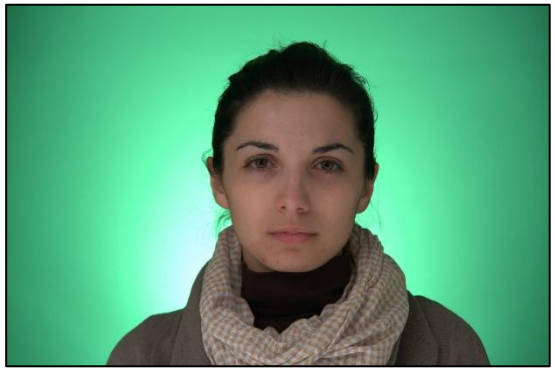
- L'immagine viene suddivisa in finestre non sovrapposte di dimensione prefissata per ogni livello della gerarchia.
- Ogni finestra viene suddivisa in patch non sovrapposte di dimensione prefissata.
- Ogni patch viene linearizzata e integrata con la posizione relativa all'interno della finestra corrispondente.
- Il modello codifica le relazioni che intercorrono tra le varie patch di una stessa finestra.
- Lungo i livelli della gerarchia le finestre vengono shiftate per catturare le connessioni tra finestre di livelli diversi.
- Le finestre vengono fuse progressivamente, riducendo la risoluzione ma aumentando il numero di canali.
- Swin ha raggiunto su ImageNet un'accuratezza pari all'87% (error rate del 13%) mostrando risultati comparabili con le CNN.

Image pre-processing

- In uno scenario reale, le immagini presentano differenze che, se non propriamente standardizzate, possono ridurre sensibilmente le prestazioni di un sistema di visione:
  - dimensione;
  - risoluzione;
  - contrasto;
  - luminosità;
  - numero di canali (RGB/grayscale);
  - pixel range ([0; 255], [0; 1]);
  - errori di etichettatura.



















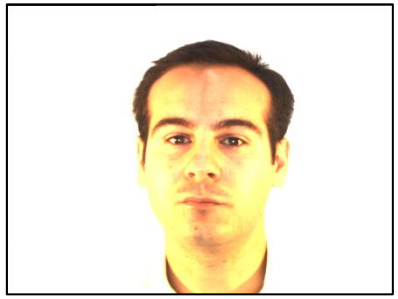


Image pre-processing (2)

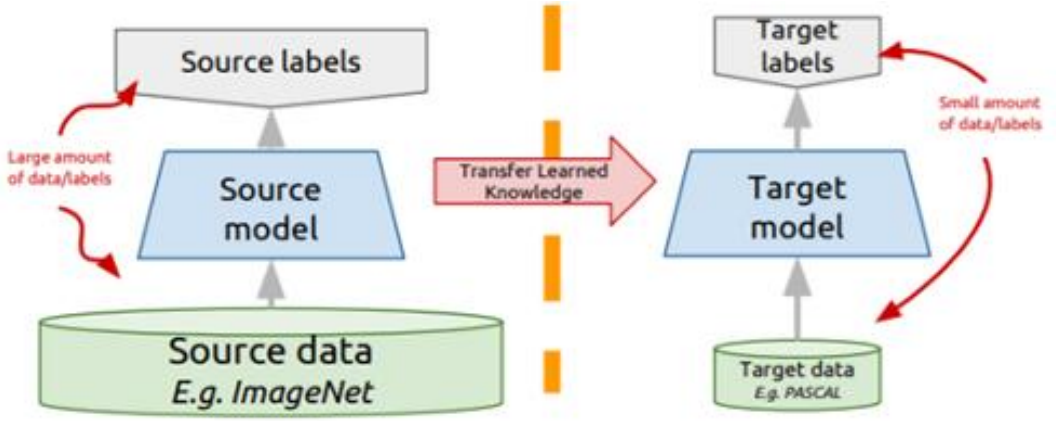
- La fase di pre-processing è fondamentale nello sviluppo di un sistema di visione.
- Non esiste una "ricetta" unica, si devono analizzare le immagini a disposizione e selezionare le operazioni più opportune.
- Operazioni comuni di pre-processing:
  - resize;
  - aspect ratio;
  - crop;
  - conversione in grayscale;
  - allineamento;
  - Normalizzazione colore.

Transfer learning

Il training di reti complesse su dataset di grandi dimensioni può richiedere giorni/settimane di tempo macchina anche se eseguito su più GPU in parallelo.

Inoltre, il training su un nuovo problema, richiede un training set etichettato di notevoli dimensioni.

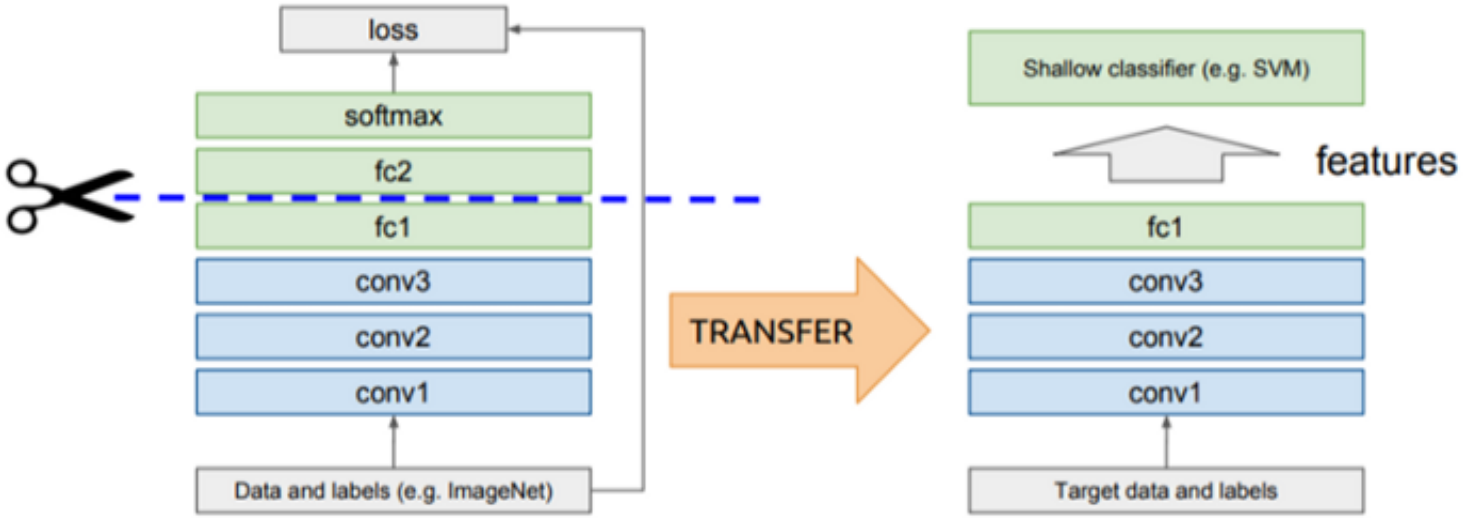
Con *transfer learning* si intende un insieme di tecniche per adattare un modello pre-addestrato (su uno specifico dominio per un determinato task) a un nuovo dominio o a un nuovo task.



Transfer learning (2)

Utilizzare un modello pre-addestrato come estrattore di features:

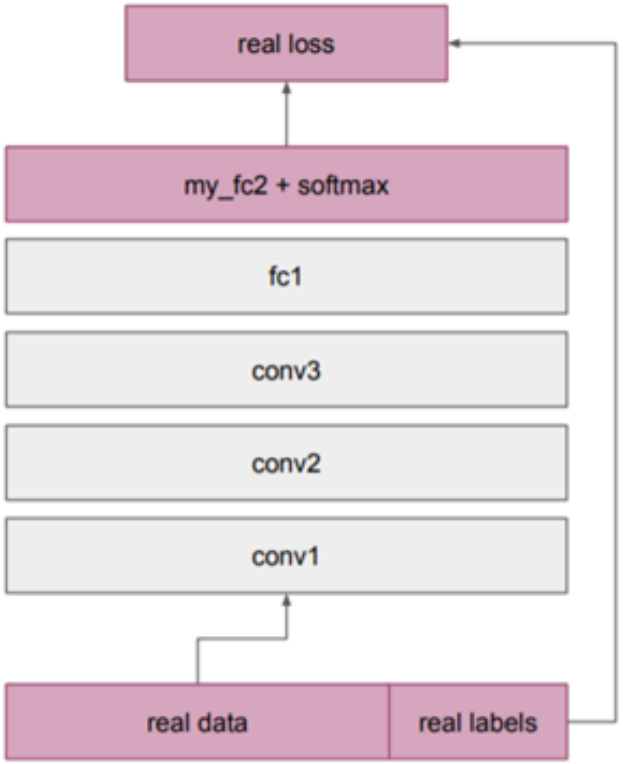
- si estraggono (da uno o più livelli intermedi) le feature generate dalla rete e si utilizzano per addestrare un classificatore tradizionale (es. SVM) a classificare i pattern del nuovo dominio applicativo.



Transfer learning (3)

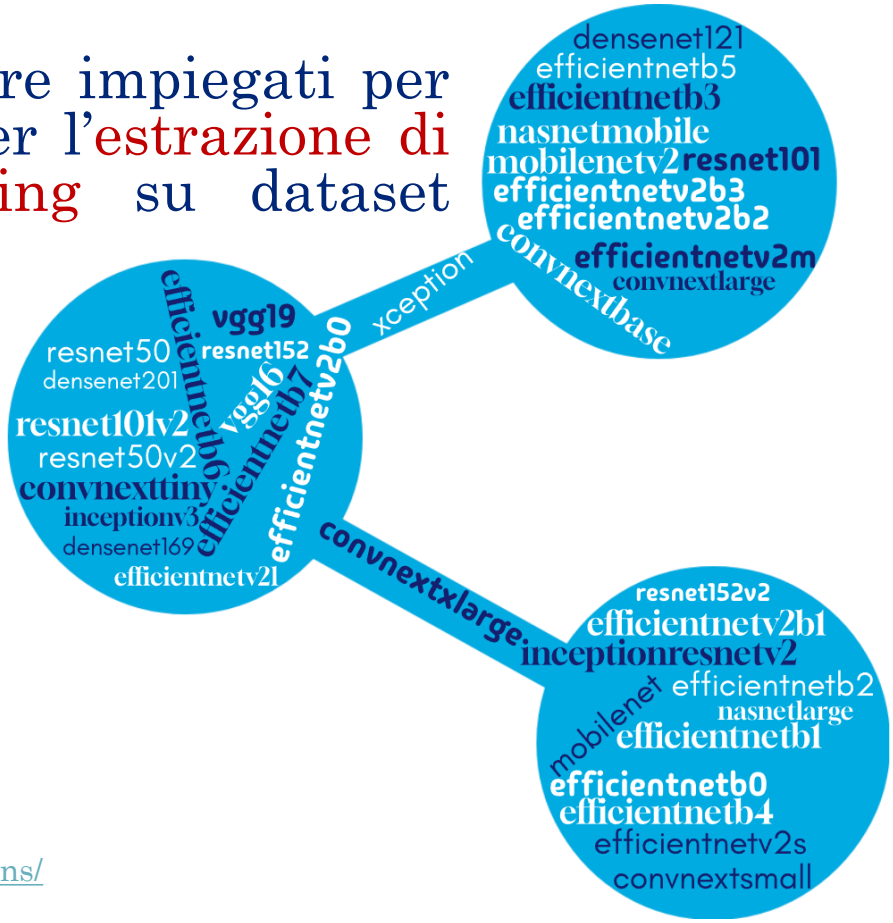
Effettuare *fine-tuning* di un modello pre-addestrato:

- si parte con una rete pre-addestrata su un problema simile;
- si rimpiazza il livello di output con un nuovo livello di output softmax (adeguando il numero di classi);
- come valori iniziali dei pesi si utilizzano quelli della rete pre-addestrata, tranne che per le connessioni tra il penultimo e l'ultimo livello i cui pesi sono inizializzati random;
- si eseguono nuove iterazioni di addestramento per ottimizzare i pesi rispetto al nuovo dataset.



Modelli disponibili - Keras

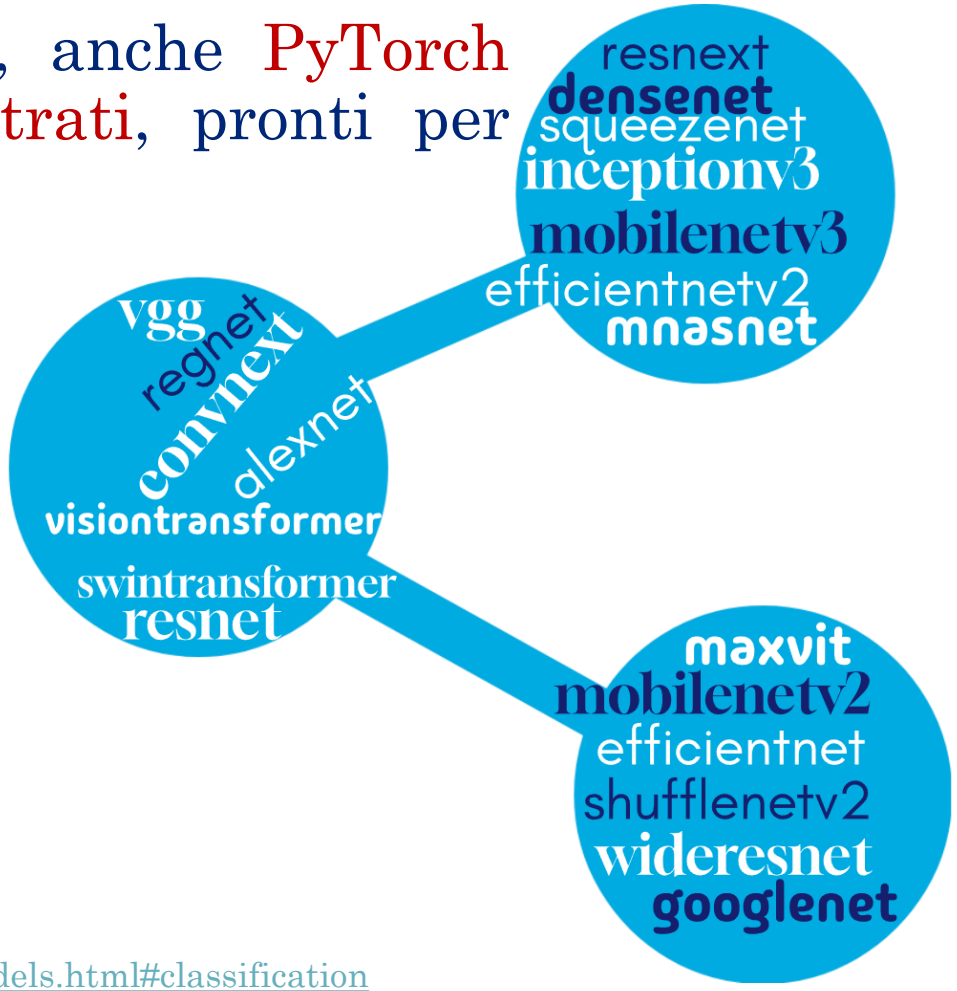
- Keras fornisce modelli all'avanguardia già addestrati su dataset di riferimento, con pesi predefiniti pronti all'uso.
- Questi modelli possono essere impiegati per compiti di classificazione, per l'estrazione di feature e per il fine-tuning su dataset specifici.



https://keras.io/api/applications/

Modelli disponibili - PyTorch

- Analogamente a Keras, anche PyTorch offre modelli pre-addestrati, pronti per essere utilizzati.



https://docs.pytorch.org/vision/main/models.html#classification

Perché servono grandi quantità di dati?

Le reti neurali moderne contengono milioni (o persino miliardi) di parametri addestrabili. Per ottenere buone prestazioni, devono vedere un numero proporzionale di esempi.

